Estimators for Measures of Presortedness

Bachelor Thesis

Supervisor:
Prof. Dr. Sebastian Wild
Head of Algorithms Group, Department of Mathematics and Computer Science
University of Marburg

Vladislav Zeynalov
(B.Sc. Computer Science)

November 18, 2025

Abstract

Adaptive sorting algorithms can significantly reduce the running time when the input array already contains some degree of order. Leveraging such opportunities in practice requires estimating the level of presortedness before sorting takes place. However, computing most presortedness metrics exactly requires scanning the entire array, which may be too expensive when data is large or arrives in streaming form. This raises the question whether global presortedness properties can be inferred accurately from small samples.

This work develops a systematic experimental framework to evaluate sampling-based estimation of presortedness. A set of data generators is defined to produce input arrays with controlled structural characteristics, independently varying value diversity (presence of duplicates) and positional order (number of runs). In addition, three order-preserving sampling strategies are examined—cluster-based sampling, stratified sampling, and a hybrid combined approach—each exposing different trade-offs between local continuity and global coverage. Several presortedness metrics are then evaluated both on full arrays and on sampled subsets.

Experiments show that some metrics remain stable under sampling, while others exhibit scale distortion or systematic bias depending on array structure and sample geometry. In particular, *structure-sensitive* metrics (such as number of runs, run entropy, oscillations, removals, and inversions) and *duplicate-sensitive* metrics (such as number of distinct elements, distinct element entropy and value-frequency entropy) are reliably predictable from samples when locality and coverage are appropriately balanced. These findings suggest that sample-based presortedness estimation is feasible and informative, supporting potential integration into adaptive sorting systems.

Declaration of Authorship

I hereby declare that I have written the present Bachelor's thesis entitled "Estimators for Measures of Presortedness" independently and that no sources or aids other than those indicated have been used. All passages taken directly or in paraphrased form from other works are clearly identified as such. This also applies to the use of secondary literature.

Marburg 18.11.2025

(Place, Date)

(Signature)

Contents

1 Introduction **6**

2 Related Work **6**

3 Preliminaries **7**

3.1 Number of Runs (R) . . . 8
3.2 Run-length Entropy (H_{runs}) . . . 8
3.3 Number of Inversions (Inv) . . . 8
3.4 Removals to Sort (Rem) . . . 9
3.5 Oscillation Count (Osc) . . . 9
3.6 Spearman Footrule Displacement (Dis) . . . 9
3.7 Hamming Distance to Sorted Order (Ham) . . . 9
3.8 Distinct Count (U) . . . 10
3.9 Value-frequency Entropy (H_{freq}) . . . 10
3.10 Distinct-element Entropy (H_0) . . . 10
3.11 Metric Normalization . . . 11

4 Data Generation **11**

4.1 Random permutation . . . 12
4.1.1 Model . . . 12
4.1.2 Algorithmic realization (pseudocode) . . . 12
4.2 Random array sampled from a pool of k values . . . 12
4.2.1 Model . . . 12
4.2.2 Algorithmic realization (pseudocode) . . . 13
4.3 Multiset permutation . . . 14
4.3.1 Model . . . 14
4.3.2 Algorithmic realization (pseudocode) . . . 14
4.4 Run-structured arrays (noise-injection method) . . . 15
4.4.1 Model . . . 15
4.4.2 Feasible positions and capacity . . . 15
4.4.3 Algorithmic realization (pseudocode) . . . 15
4.4.4 Distribution and order of elements . . . 17
4.4.5 Limitations . . . 18
4.5 Run-structured arrays (permutation–segmentation method) . . . 18
4.5.1 Model . . . 18
4.5.2 Segment construction and ordering . . . 18
4.5.3 Algorithmic realization (pseudocode) . . . 19

4.5.4 Distribution and order of elements . . . 20
4.5.5 Limitations . . . 21
4.6 Run-structured arrays (sorted-base–segmentation method) . . . 21
4.6.1 Model . . . 21
4.6.2 Segment construction and ordering. . . . 21
4.6.3 Algorithmic realization (pseudocode) . . . 21
4.6.4 Distribution and order of elements . . . 23
4.6.5 Limitations . . . 23
4.7 Overview . . . 24

5 Sampling Methodology **24**
5.1 Cluster-Based Sampling . . . 25
5.1.1 Model . . . 25
5.1.2 Algorithmic realization (pseudocode). . . . 26
5.1.3 Coverage analysis (cluster length scaling). . . . 27
5.1.4 Limitations. . . . 29
5.2 Stratified Sampling . . . 29
5.2.1 Algorithmic realization (pseudocode) . . . 30
5.3 Coverage analysis (stratum-length scaling). . . . 32
5.3.1 Limitations. . . . 32
5.4 Combined Sampling (stratified–clustered) . . . 33
5.4.1 Model . . . 33
5.4.2 Algorithmic realization (pseudocode). . . . 33
5.4.3 Coverage analysis (stratum-length scaling). . . . 34
5.4.4 Limitations. . . . 35
5.5 Overview . . . 35

6 Experiment Design **36**
6.1 Aim and Scope . . . 36
6.2 Data Generation Configurations . . . 37
6.2.1 Random Permutation (baseline) . . . 37
6.2.2 Random Array from a k-Value Pool (duplicate-controlled i.i.d.) . . . 37
6.2.3 Multiset Permutation (exactly u distinct values) . . . 37
6.2.4 Run-Structured Arrays (exactly R strictly increasing runs) . . . 38
6.3 Sampling Methods and Configurations . . . 38
6.3.1 Cluster-Based Sampling . . . 38
6.3.2 Strat-Based Sampling . . . 39
6.4 Experiment Organization and Data Layout . . . 39
6.4.1 Stage 1: Full-Array Generation . . . 40

6.4.2 Stage 2: Sampling from Full Arrays . . . 40
6.4.3 Formation of Analysis Datasets . . . 41
6.5 Analytical Methodology . . . 41

7 Experiment Results **42**
7.1 Random-Combined: Results and Analysis . . . 43
7.1.1 Cluster Sampling . . . 43
7.1.2 Stratified Sampling . . . 44
7.1.3 Combined Sampling . . . 45
7.2 Summary. . . . 47
7.3 Run-Combined: Results and Analysis . . . 48
7.3.1 Cluster Sampling . . . 48
7.3.2 Stratified Sampling . . . 50
7.3.3 Combined Sampling . . . 52
7.4 Summary. . . . 57

8 Discussion **57**
8.1 Overview . . . 57
8.2 Structure–Sensitive Metrics (Run–Combined) . . . 58
8.3 Duplicate–Sensitive Metrics (Random–Combined) . . . 58

9 Conclusion **59**

10 Future Work **59**
10.1 Data Generation Extensions . . . 59
10.2 Sampling Performance under Varying Dataset Sizes . . . 59
10.3 Adaptive Sampling . . . 60

A Appendix A: Pseudocode for Presortedness Measures **61**

A Appendix B: Helper Routines for Data Generation **65**

Bibliography **69**

1 Introduction

Adaptive sorting algorithms exploit existing structure in the input to reduce running time below the $\Theta(n \log n)$ bound. The degree to which an array is already ordered—its *presortedness*—can substantially influence performance. For example, natural mergesort, Timsort and the recent Powersort algorithm adapt their running time to the number and lengths of monotone runs (Knuth, 1998; Peters, 2002, 2004; Munro and Wild, 2018). A large body of work has therefore focused on presortedness as a quantitative characterization of how much exploitable structure an input contains (Estivill-Castro and Wood, 1992). Computing most presortedness metrics exactly (e.g., number of runs or inversion count) requires at least linear time in the full array length, which may be undesirable for large-scale data.

This motivates the central question of this work:

> **Can global presortedness be inferred reliably from small, order-preserving samples of the input array?**

If accurate and computationally cheap presortedness estimation is possible from small samples, sorting systems could adaptively select strategies, tune merge policies, or avoid unnecessary work, all without scanning the entire input. However, sampling introduces a tension between *locality* (capturing fine-scale structural patterns such as run boundaries) and *coverage* (representing the global characteristics of the array). The way a sample is constructed therefore critically affects the stability and informativeness of sample-based metric estimates.

To study this systematically, the analysis employs a controlled experimental framework built on two components. First, synthetic data families with tunable structure are constructed, varying independently along value-space diversity (controlled duplication and support size) and position-space organization (arrays with a prescribed number of monotone runs). This enables precise control over both distributional and order-based properties of the input. Second, three order-preserving sampling strategies—cluster sampling, stratified sampling, and a hybrid combined method—are evaluated, each parameterized over a range of locality and coverage scales.

Presortedness metrics are computed both on full arrays and on their samples, and we assess sample–array agreement using correlation, scale consistency, and bias. By comparing across structural regimes ranging from fully random permutations to strongly run-dominated arrays, we identify which metrics are robust under sampling, which sampling geometries best preserve metric meaning, and how these patterns depend on the underlying arrangement of values.

Overall, this study provides a principled basis for *sampling-based presortedness estimation*, offering insight into how adaptive sorting and data systems may benefit from fast presortedness assessment without accessing the entire input.

2 Related Work

The idea that input order can be exploited to beat the $\Theta(n \log n)$ bound on *average* or *worst* cases traces back to foundational theoretical and systems work (Knuth, 1998; Bentley and McIlroy, 1993). A substantial line of research introduced the notion of *adaptive sorting*, where runtime depends not only on n but also on how far the input is from being sorted, formalized as *presortedness* (Estivill-Castro and Wood, 1992; Mannila, 1985; Petersson and Moffat, 1995; Levcopoulos and Petersson, 1993).

An important notion in the study of adaptive sorting is optimality with respect to a given presortedness measure. Intuitively, a sorting algorithm is considered well-adapted to a measure M if its running

time improves in direct relation to the amount of order captured by M. Mannila formalized this idea and showed how presortedness measures can be used to characterize when an algorithm is optimally exploiting existing order in the input (Mannila, 1985). This perspective provides a principled basis for comparing adaptive algorithms beyond traditional worst-case analysis.

A further theoretical unification was developed by Estivill–Castro, Mannila, and Wood, who examined the relationship between presortedness measures and *right-invariant metrics* on permutations (Estivill-Castro et al., 1993). Their work shows that many presortedness measures used in computer science (such as inversions, exchanges, displacement, Hamming distance, and removals to sort) can be viewed as instances of—or extensions to—right-invariant metrics. They characterize when a presortedness measure can be extended to such a metric (*normal* measures), and conversely, when a right-invariant metric can serve as a presortedness measure.

Several practical adaptive sorting algorithms specialize in particular presortedness measures. Natural mergesort, already described by Knuth (Knuth, 1998, §5.2.4), is R-optimal in the sense that its running time adapts to the number and lengths of monotone runs in the input. Modern production variants such as Timsort (used in Python and Java) (Peters, 2002, 2004) and the more recent Powersort (Munro and Wild, 2018) refine this idea through improved run merging strategy. Insertion sort, in turn, is *inversion-optimal*, as shown by Mehlhorn (Mehlhorn, 1979). These cases demonstrate that different structural forms of partial order naturally lead to different optimal adaptive sorting strategies.

The measure $\mathrm{Rem}(X)$, equivalent to $n-\mathrm{LIS}(X)$, has been used to characterize nearly-sorted inputs in empirical studies (Cook and Kim, 1980). Efficient algorithms for computing Rem and LIS in $O(n \log n)$ time were developed in classical works (Dijkstra, 1980) and (Dewar et al., 1982).

Finally, inputs containing *many duplicate keys* require separate consideration. Practical sorting implementations handle duplicates explicitly, typically by using stable merging or by employing three-way partitioning (Bentley and McIlroy, 1993; Sedgewick, 1998). At a theoretical level, the presence of duplicates changes the combinatorial structure of the sorting problem: the effective search space becomes a multiset permutation rather than a permutation of distinct elements. This affects both algorithmic optimality criteria and the behavior of several presortedness measures. Recent analysis has shown that variants of Quicksort that use adaptive pivot selection and partitioning tuned for value multiplicities can achieve optimal comparison counts on distributions with sufficiently frequent duplicate values (Wild, 2017), addressing a conjecture originating from work by Sedgewick and Bentley (Sedgewick and Bentley, 2002). These results indicate that the presence of duplicates can influence both sorting performance and the behavior of presortedness metrics.

Taken together, prior research establishes presortedness as a *broad structural concept*, with different measures capturing distinct types of order (run-based, inversion-based, displacement-based, and duplicate-sensitive). The present work builds on this foundation by investigating whether such structural properties can be *reliably estimated from small samples*.

3 Preliminaries

Before introducing the data–generation models and sampling strategies used in the experimental evaluation, we first establish the presortedness measures that serve as the primary quantitative features throughout this work. These measures capture different ways in which an input array may depart from sorted order. For each metric, we provide a concise mathematical definition together with the computational cost required to evaluate it. Pseudocode implementations corresponding to these definitions are supplied in Appendix A.

For a broader theoretical treatment of presortedness measures and their structural relationships, see

the survey by Estivill–Castro and Wood (Estivill-Castro and Wood, 1992).

3.1 Number of Runs (R)

This measure captures how much of the array is already organized into monotone increasing structure. When R is small relative to n, the array consists of a few long increasing blocks, indicating strong global order. Conversely, when R is large—especially approaching the expected value of $n/2$ in a random permutation—the array behaves close to completely disordered. Values of R much larger than $n/2$ indicate that decreasing trends dominate the ordering, positioning the sequence closer to being reverse–sorted. Thus, R provides a coarse but powerful indicator of the global monotonic structure in the input.

Definition. Let $a = (a_1, \ldots, a_n)$. A *run* is a maximal contiguous strictly increasing subsequence. The number of runs is:

$$R(a) \;=\; \big|\{\, i \mid 1 \le i < n,\; a_{i+1} < a_i \,\}\big| + 1.$$

Complexity. One left-to-right scan: time $O(n)$, space $O(1)$.

3.2 Run-length Entropy ($H_{\mathbf{runs}}$)

While R counts how many runs exist, it does not describe their length distribution. H_{runs} captures how evenly or unevenly the run lengths are allocated. If one run dominates, the entropy is low; if the runs are roughly equal in length, the entropy increases. This measure therefore distinguishes, for example, one long sorted prefix from many moderate-sized segments.

Definition. Let the run lengths be $r_1, \ldots, r_R$ with $\sum_{k=1}^{R} r_k = n$, and $p_k = r_k/n$.

$$H_{\text{runs}}(a) \;=\; -\sum_{k=1}^{R} p_k \ln p_k.$$

Complexity. Run detection $O(n)$, entropy computation $O(R)$; total time $O(n)$, space $O(R)$.

3.3 Number of Inversions (Inv)

Inversions measure global disorder: each inversion counts a pair of elements that appear out of order. An array is sorted exactly when Inv=0, and maximally disordered when every earlier index contains a larger value than every later one.

Definition.

$$\mathrm{Inv}(a) = \big|\{(i,j) \mid 1 \le i < j \le n,\; a_i > a_j\}\big|.$$

Complexity. Computed using mergesort-based counting: time $O(n \log n)$, space $O(n)$.

3.4 Removals to Sort (Rem)

This measure quantifies how many elements would need to be deleted to make the sequence sorted. Equivalently, it measures how far the sequence is from containing a long monotone subsequence.

Definition. Let $\mathrm{LIS}(a)$ denote the length of the longest non-decreasing subsequence.

$$\mathrm{Rem}(a) = n - \mathrm{LIS}(a).$$

Complexity. Patience sorting with binary search: time $O(n \log n)$, space $O(n)$.

3.5 Oscillation Count (Osc)

While the number of runs reflects how often monotonic direction resets, it does not distinguish whether those resets occur in large structured blocks or as rapid fluctuations. The oscillation count measures the local frequency at which the sequence changes direction, from increasing to decreasing or from decreasing to increasing. Arrays that frequently alternate direction—even if their values are close to sorted—receive high oscillation scores, whereas smooth monotone or piecewise monotone sequences exhibit low oscillation. Thus, oscillation isolates small-scale directional variation that is not captured directly by global run counts.

Definition.

$$\mathrm{Osc}(a) = \big|\{\, i \mid 2 \le i \le n-1,\ (a_{i-1} - a_i)(a_i - a_{i+1}) < 0\}\big|.$$

Complexity. Single pass: time $O(n)$, space $O(1)$.

3.6 Spearman Footrule Displacement (Dis)

This metric measures how far elements move under stable sorting. If most elements already lie close to their final sorted positions, Dis remains small even when the array is not fully sorted.

Definition. Let $\pi(i)$ be the index of a_i in the stably sorted array.

$$\mathrm{Dis}(a) \;=\; \sum_{i=1}^{n} |\, i - \pi(i)\,|.$$

Complexity. Stable sort + index matching: time $O(n \log n)$, space $O(n)$.

3.7 Hamming Distance to Sorted Order (Ham)

This measure counts how many values are in the "wrong" position compared to their final sorted position. It is sensitive to whether a value is correct *exactly where it is*, unlike displacement.

Definition.

$$\mathrm{Ham}(a) = \big|\{\, i \mid a_i \neq \mathrm{sorted}(a)_i\}\big|.$$

Complexity. Requires sorting a copy: time $O(n \log n)$, space $O(n)$.

3.8 Distinct Count (U)

Distinct count expresses the size of the value support: how many unique values appear in the array. This measure determines the degree to which value multiplicity influences the behavior of sorting algorithms and presortedness metrics. When U is close to n, the array behaves like a permutation of distinct keys—not necessarily random, and potentially exhibiting its own structure such as runs; when U is small, large blocks of equal values dominate the structure. In the latter case, many ordering–based metrics (inversions, displacement, run structure) behave differently, since equal values may be considered already "sorted" with respect to one another. Therefore, U plays a central role in distinguishing duplicate–driven structure from position–driven structure.

Definition.

$$U(a) = \big|\{\, a_i \;\mid\; 1 \le i \le n\}\big|.$$

Complexity. Computed via hashing: expected time $O(n)$, space $O(U)$.

3.9 Value-frequency Entropy ($H_{\mathbf{freq}}$)

This measures how evenly duplicates are distributed. Uniform repetition of each value maximizes entropy, while one value dominating the array minimizes it.

Definition. Let $f(v)$ be the number of occurrences of value v, and $p(v) = f(v)/n$.

$$H_{\text{freq}}(a) = -\sum_{v} p(v) \ln p(v).$$

Complexity. Count frequencies + entropy sum: expected $O(n)$ time, $O(U)$ space.

3.10 Distinct-element Entropy (H_0)

This is the Hartley entropy of the value support: it measures the effective size of the set of distinct values on a logarithmic scale. Unlike U, which grows linearly with distinctness, H_0 represents distinctness in information units (nats). In this context, H_0 reflects how "broad" the value space is relative to the array length. Low H_0 indicates that duplicates are highly concentrated and the value distribution is narrow, while high H_0 indicates broad support approaching permutation-like behavior.

Definition.

$$H_0(a) = \ln U(a).$$

Complexity. Dominated by computing U: expected time $O(n)$, space $O(U)$.

3.11 Metric Normalization

To compare arrays of different lengths and, crucially, to compare *sample-based* estimates to *full-array* ground truth, we map each metric to $[0, 1]$ by dividing by its maximum attainable value under the given conditions. This preserves order (monotonicity) while putting heterogeneous measures on a common scale suitable for statistical analysis.

Table 1: Normalization formulas used to scale presortedness metrics to the $[0, 1]$ range.

Metric	**Normalization to** $[0, 1]$
Inversions Inv	$\mathrm{Inv}/\binom{n}{2}$
Runs R	$(R-1)/(n-1)$
Run entropy H_{runs}	$H_{\mathrm{runs}}/\ln R$
Distinct count U	$(U-1)/(n-1)$
Value-frequency entropy H_{freq}	$H_{\mathrm{freq}}/\ln U$
Distinct-element entropy H_0	$H_0/\ln n$
Removals Rem	$\mathrm{Rem}/(n-1)$
Oscillation Osc	$\mathrm{Osc}/(n-2)$
Spearman footrule Dis	$\mathrm{Dis}/\left(\begin{array}{l} n^2/2, \text{ if } n \text{ even} \\ (n^2-1)/2, \text{ if } n \text{ odd} \end{array}\right)$
Hamming distance Ham	Ham/n

4 Data Generation

The data–generation models used in this study provide a controlled framework for evaluating how presortedness behaves under varying forms of structural regularity and disorder. The primary objective is to define input families that (i) span both common and adversarial regimes for presortedness metrics, (ii) possess clear structural parameters that can be systematically tuned, and (iii) have well–defined and analyzable properties with respect to length, value range, run count, and distinctness. The underlying hypothesis is that presortedness measures computed on small, order–preserving samples can serve as reliable predictors of global sortedness and, consequently, as features for selecting and tuning adaptive sorting algorithms. To examine both the sensitivity and the limitations of this hypothesis, *value–space* variability (e.g., duplicates) is separated from *position–space* structure (e.g., runs), while additional models combine these aspects to varying degrees.

Four principal classes of input models are employed:

1. **Random permutation** acts as a neutral baseline: it exhibits maximal positional disorder with no duplication and no run structure. This model establishes the reference behavior of presortedness metrics in the absence of imposed regularity.

2. **Random array from a k–value pool** isolates the role of limited value diversity. The support size k controls duplication while keeping positions i.i.d.; this family stresses metrics that depend on collisions and ties (e.g., number of distinct elements, distinct entropy, and value–frequency entropy).

3. **Multiset permutation** controls duplication while maintaining positional randomness by fixing the support size u (the exact number of distinct labels) and randomizing both the multiplicity profile and the final ordering. This model enables analysis of the same value–diversity metrics as the k–pool family—number of distinct elements, distinct entropy, and value–frequency entropy—under a fixed support size.

4. **Run–structured arrays** introduce explicit positional order by prescribing the number of strictly increasing runs. Three complementary constructions are considered—noise–injection, permutation–segmentation, and sorted–base–segmentation—which differ in the placement of run boundaries and in the distribution of values within runs. These models are used to evaluate metrics that directly measure order (runs, run entropy, inversions, oscillations, displacement).

Each generator includes formally stated preconditions and correctness guarantees, and is parameterized so that experimental grids can vary a single structural parameter (e.g., k, u, or R) at a time, enabling controlled and reproducible comparisons across sampling strategies and metric definitions. Pseudocode for all auxiliary routines used by the generation methods is provided in Appendix B.

4.1 Random permutation

4.1.1 Model

Uniform distribution over all permutations of $\{1, \ldots, n\}$; no duplicates, every ordering has probability $1/n!$.

4.1.2 Algorithmic realization (pseudocode)

```
procedure RandomPermutation(n):

    # Preconditions:
    #   n > 0. Values are 1..n.

    A <- [1, 2, ..., n]
    ShuffleUniformly(A)
    return A
```

Complexity. Time $O(n)$; extra space $O(1)$ beyond the output array.

Correctness guarantees **Claim 1 (length and value range).** The procedure initializes an integer array $1, \ldots, n$ and applies an in–place uniform shuffle. No insertions/deletions occur; hence $|A| = n$. Shuffling permutes these positions bijectively; therefore every value appears exactly once and A is a permutation of $\{1, \ldots, n\}$. □

4.2 Random array sampled from a pool of k values

4.2.1 Model

Let the target array length be $n \geq 0$ and the value range be the integer interval $[a, b]$ with width $W = b - a + 1$. The generator proceeds in two independent stages:

1. **Pool selection (support).** Draw a set $P \subseteq [a, b]$ of size $k' = \min\{k, W\}$ uniformly without replacement; each $\binom{W}{k'}$ subset is equally likely.
2. **Array sampling (labels).** Produce $A = (A_1, \ldots, A_n)$ by i.i.d. draws from P with replacement. Given P, $\Pr[A_i = x] = 1/k'$ for every $x \in P$.

Hence the array entries are i.i.d. conditional on the random support P. If $k \geq W$ this reduces to the uniform i.i.d. model on $[a, b]$.

Distributional properties

- **Occupancy vector.** Conditional on $P = \{v_1, \ldots, v_{k'}\}$, the multiplicities C_j are multinomial: $\mathrm{Mult}\big(n; \frac{1}{k'}, \ldots, \frac{1}{k'}\big)$.
- **Expected distinct.** $\mathbb{E}[\mathrm{distinct}(A) \mid P] = k'\big(1 - (1 - \frac{1}{k'})^n\big)$.

4.2.2 Algorithmic realization (pseudocode)

```
procedure RandomArrayFromPool(n, a, b, k):

    # Preconditions:
    #   n > 0
    #   a <= b
    #   k > 0

    # Range width and effective pool size
    W  <- (b - a + 1)
    k' <- min(k, W)                # clamp k to available distinct values

    # Build pool P of size k'

    # uniformly sample k' unique indices from [0..W).

    J <- SampleWithoutReplacement(W, k')
    P <- new array of length k'
    for t <- 0 to k' - 1 do
        P[t] <- a + J[t]           # map indices to actual values in [a..b]
    end for

    # Sample the output array i.i.d. from P
    A <- new array of length n
    for i <- 0 to n - 1 do
        t <- UniformInteger(0, k' - 1)
        A[i] <- P[t]
    end for
    return A
```

Complexity. Pool construction $O(k')$ time/space (expected); array sampling $O(n)$ time and $O(1)$ extra space.

Correctness guarantees **Claim 1 (length).** The algorithm writes exactly n entries; thus $|A| = n$.
Claim 2 (value range). First, a support $P \subseteq [a, b]$ of size $k' = \min\{k, b - a + 1\}$ is drawn uniformly without replacement. Then each entry of A is sampled i.i.d. *with replacement* from P. Hence every $A_i \in P \subseteq [a, b]$.
Claim 3 (distinct upper bound and occupancy law). Because sampling is from P with replacement, $\text{distinct}(A) \leq |P| = k' \leq k$. Conditional on P, the multiplicity vector over P is multinomial with equal cell probabilities $1/k'$. $\square$

4.3 Multiset permutation

4.3.1 Model

Given a target length n and a required number of distinct labels u ($\texttt{base}, \texttt{base} + 1, \ldots, \texttt{base} + u - 1$), first draw a *uniform positive composition* $x = (x_1, \ldots, x_u)$ of n into u parts ($x_i \geq 1$, $\sum_{i=1}^{u} x_i = n$). Materialize the multiset and apply a uniform shuffle.

Distributional properties

- **Profile law.** x is uniform on positive compositions; coordinates exchangeable with $\mathbb{E}[x_i] = n/u$.
- **Distinctness control.** The distinct count equals u deterministically; only multiplicity balance is random.

4.3.2 Algorithmic realization (pseudocode)

```
procedure MultisetPermutationExactProfile(n, u, base):

    # Preconditions:
    #   n > 0
    #   u >= 1
    #   n >= u            # ensures each label appears at least once
    #   Labels used: base, base+1, ..., base+u-1

    # Draw a positive composition x = (x1,...,xu) of n into u parts,
    # chosen uniformly among all such compositions.
    x <- RandomPositiveProfile(n, u)

    # Materialize the multiset and shuffle uniformly.
    A <- new array of length n
    pos <- 0
    for i <- 0 to u-1 do
        for j <- 1 to x[i] do
            A[pos] <- base + i
            pos <- pos + 1
```

```
        end for
    end for

    ShuffleUniformly(A)
    return A
```

Complexity. Sampling x: $O(u)$. Building and shuffling: $O(n)$ time, $O(1)$ extra space (plus $O(u)$ to store x).

Correctness guarantees. **Claim 1 (length).** The routine samples a positive composition $(x_1, \ldots, x_u)$ of n and then emits exactly x_i copies of label i before shuffling. Therefore $\sum_{i=1}^{u} x_i = n$ equals the array length.

Claim 2 (value range). Labels are drawn from $\{\texttt{base}, \ldots, \texttt{base} + u - 1\}$; thus all values lie in that interval (and in $[1..n]$ in the experiments).

Claim 3 (exact distinct count). Because the composition is *positive*, each $x_i \geq 1$, so every label occurs at least once. Hence $\text{distinct}(A) = u$ deterministically. □

4.4 Run-structured arrays (noise-injection method)

4.4.1 Model

Start from a strictly increasing base $a[i] = i + 1$ ($0 \leq i \leq N - 1$, one run). Choose $R - 1$ *non-adjacent* internal indices i from a feasible window and at each such index replace $a[i]$. Each replacement creates exactly one new run boundary; with $R - 1$ drops the output has R strictly ascending runs.

4.4.2 Feasible positions and capacity

A strict drop at an internal index $i \in \{1, \ldots, N - 2\}$ is feasible if there exists a replacement value in

$$\big[\texttt{minValue},\ \min\{a[i-1], a[i+1]\} - 1\big].$$

For the strictly increasing base $a[i] = i + 1$ this simplifies to $a[i - 1] > \texttt{minValue}$, i.e.

$$i > \texttt{minValue} \quad \Rightarrow \quad i \in \{\max(1, \texttt{minValue} + 1), \ldots, N - 2\}.$$

Over any contiguous block of feasible indices of length L, the maximum number of non-adjacent drops (the *capacity*) equals $\lceil L/2 \rceil$; requesting more drops is infeasible. Non-adjacent placement prevents interactions between drops: each modified index i creates exactly one new boundary $a[i - 1] > a[i]$, and because no two drops are adjacent, their feasibility intervals and run effects do not interfere. Values inside runs remain strictly increasing; equal values may occur across different runs. Consequently, with $R - 1$ non-adjacent feasible drops the output has exactly R strictly ascending runs.

4.4.3 Algorithmic realization (pseudocode)

```
procedure GenerateRunsNoise(n, R, minValue):

    # Preconditions:
```

```
#   n >= 1
#   R >= 1
#   minValue >= 0

# Base array: strictly increasing (one run)
A <- new array of length n
for i <- 0 to n-1 do
    A[i] <- i + 1
end for

dropCount <- R - 1
if dropCount == 0 then
    return A
end if

# Determine feasible index window
first <- max(1, minValue + 1)
last  <- n - 2

# Select exactly dropCount non-adjacent indices uniformly
S <- ChooseNonAdjacentInRange(first, last, dropCount)

# Apply drops
for each i in S do
    hi <- A[i-1] - 1
    v  <- UniformInteger(minValue, hi)
    A[i] <- v
end for

return A
```

Complexity. Building the base $O(N)$, constructing the chessboard set $O(N)$, pruning and applying $R-1$ drops $O(R)$. Overall $O(N)$ time and $O(N)$ output space.

Correctness guarantees. Claim 1 (length and range). The method starts from the strictly increasing base $[1, \ldots, N]$ and overwrites values only at selected internal indices; hence no insertions/deletions occur and $|A| = N$. At each selected index i the new value is drawn from $[\texttt{minValue}, \min\{A_{i-1}, A_{i+1}\} - 1] \subseteq [\texttt{minValue}, \texttt{maxValue}]$; all other entries retain their original values. Therefore every output value lies in the prescribed range.

Claim 2 (chessboard feasible set and non-interference). Let the feasible window of internal indices be $[i_{\min}, i_{\max}]$ where feasibility requires $\min\{A_{i-1}, A_{i+1}\} - 1 \geq \texttt{minValue}$. From this window the algorithm first forms a *chessboard* superset

$$S \;=\; \{\, i_{\min},\, i_{\min}{+}2,\, i_{\min}{+}4,\, \ldots,\, \leq i_{\max} \,\},$$

i.e., indices of one fixed parity only. Any two elements of S are separated by at least one untouched neighbor. The target number of drops is obtained by pruning S uniformly at random to the desired

size. Consequently, the final update set $T \subseteq S$ is *non-adjacent* by construction. Because neighbors of every $i \in T$ remain unmodified, each overwrite affects only the local pair $(i-1, i)$ and cannot invalidate feasibility at any other $j \in T$.

Claim 3 (run count). Initially the array is strictly increasing (one run). For every $i \in T$, the new value is strictly less than A_{i-1}, creating a strict drop $A_{i-1} > A_i$. Because T is non-adjacent (by the chessboard construction), every strict drop is isolated and contributes one new run boundary. With $|T| = R - 1$ overwrites the output therefore has exactly R strictly increasing runs. □

4.4.4 Distribution and order of elements

Figure 1 shows the index–value plot for an array generated by the noise–injection method. The x-axis represents the index and the y-axis the corresponding array value. In the absence of modifications, the baseline $y = i+1$ forms a strictly increasing diagonal "roof". Noise is introduced only at selected indices by *lowering* their values to the interval $[\texttt{minValue}, i]$. This yields the following visible structure:

- **Triangular envelope.** Points at untouched indices lie exactly on the diagonal $y = i + 1$. At a modified index i, the value is redrawn uniformly from $[\texttt{minValue}, i]$, i.e., somewhere between the bottom of the plot and the current index level. Because the admissible range grows linearly with i, the union of all admissible vertical ranges across indices forms a widening, triangular region beneath the diagonal: shallow drops on the left, potentially very deep drops near the right end.

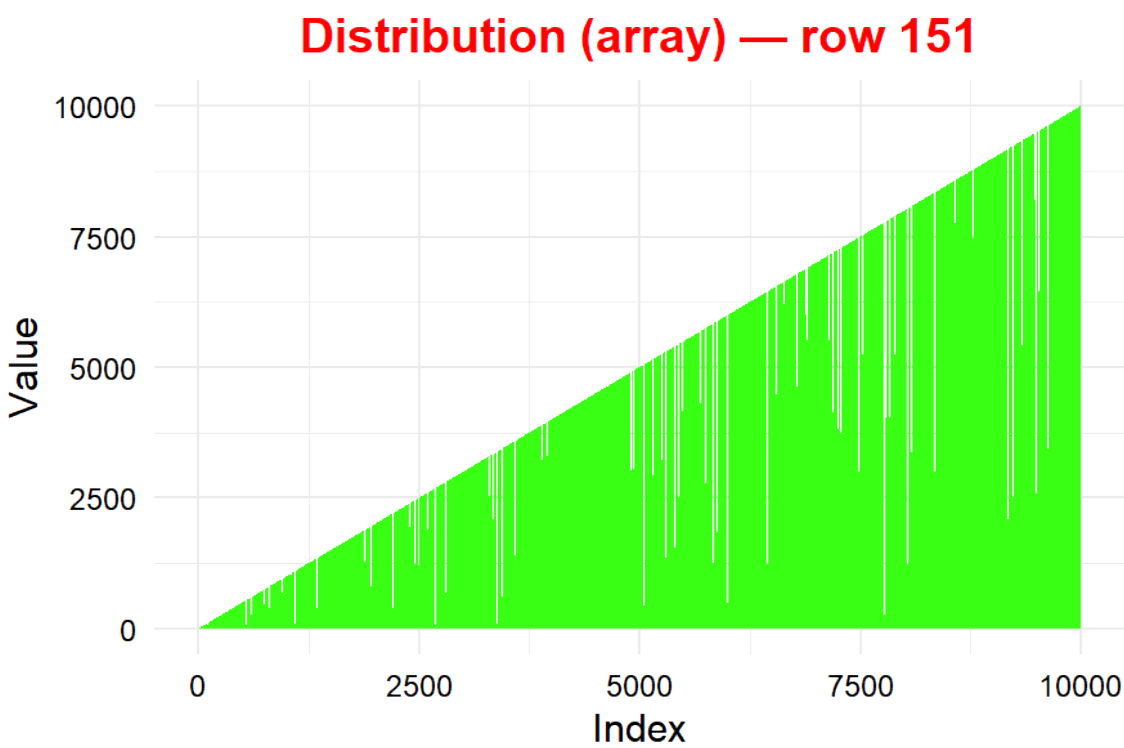


Figure 1: Noise–injection method ($N = 10{,}000$): index–value distribution for $R = N/10$.

- **Run interiors and boundaries.** Within each run the array remains strictly increasing, so points trace the diagonal without interruptions. Every modification creates a strict drop $A_{i-1} >$

A_i and the sequence resumes the diagonal at $i+1$; visually this is an isolated vertical descent at the modified index (a "pit") followed by an immediate return to the diagonal. Because modifications are enforced to be non-adjacent via the chessboard candidate set and subsampling, these pits are separated by at least one unmodified index; boundaries are thus spaced and cannot merge.

- **Feasible-window effect.** With zero-based indices the feasible window is $[\texttt{firstIdx}, N-2]$ where $\texttt{firstIdx} = \max\{1, \texttt{minValue}+1\}$. Hence the very beginning and the last index are never modified, concentrating visible boundaries in the interior of the plot.

4.4.5 Limitations

The noise–injection scheme generates run–structured arrays efficiently, but the construction induces specific structural biases. In particular:

- **Feasible-window bias.** No drops can occur at indices $0, 1$ (for $\texttt{minValue} = 1$) or at $N-1$, and in general not before $\texttt{firstIdx}$ or after $N-2$. This skews boundary placement away from the edges compared to models that allow edge-adjacent drops.
- **Index-dependent value range.** A drop at index i draws uniformly from $[\texttt{minValue}, i]$. Hence the marginal distribution of values depends on the index: deeper drops are more likely later in the array.
- **Induced run-length distribution.** Run lengths emerge as gaps between selected drop positions. Because drops are restricted to a spaced candidate set and sampled uniformly within that set, the resulting run-length distribution is neither geometric nor controlled by a direct parameter. Short runs can occur, but extremely small runs are less frequent and their positioning is not uniform. Moreover, since values inside each run remain strictly increasing, *descending runs cannot be produced* under this construction.
- **Strictly increasing run interiors.** If a workload requires duplicates inside runs (non-decreasing runs), this model cannot generate them; equal values occur only across different runs.

4.5 Run-structured arrays (permutation–segmentation method)

4.5.1 Model

An array with exactly R strictly ascending runs is generated by (i) drawing a uniformly random permutation of $\{1, \ldots, N\}$; (ii) partitioning the index range into R contiguous, positive-length segments whose lengths sum to N; (iii) sorting each segment in ascending order; and (iv) reordering the segments by strictly *descending* segment maxima before concatenation.

4.5.2 Segment construction and ordering

Let the segment lengths be $L_1, \ldots, L_R$ with each $L_r \geq 1$ and $\sum_{r=1}^{R} L_r = N$. Define the start and end indices

$$S_r \;:=\; 1 + \sum_{t<r} L_t, \qquad E_r \;:=\; S_r + L_r - 1,$$

and the within-segment extrema (after the local ascending sort)

$$\min_r \;:=\; \min\{\, a[j] : j \in [S_r, E_r] \,\}, \qquad \max_r \;:=\; \max\{\, a[j] : j \in [S_r, E_r] \,\}.$$

Reorder the R segments so that their maxima are *strictly decreasing*:

$$\max_{\pi(1)} > \max_{\pi(2)} > \cdots > \max_{\pi(R)}.$$

After concatenation in the order $\pi(1), \ldots, \pi(R)$, every boundary is a strict drop, because for each $t = 1, \ldots, R-1$,

$$a[E_{\pi(t)}] = \max_{\pi(t)} > \max_{\pi(t+1)} \geq a[S_{\pi(t+1)}],$$

while the interior of each segment remains strictly ascending by construction. Therefore the output contains exactly R strictly ascending runs.

4.5.3 Algorithmic realization (pseudocode)

```
procedure GenerateRunsFromPermutationSortedSegments(n, R):

    # Preconditions:
    #   1 <= R <= n

    # Random permutation of values 1..n
    P <- RandomPermutation(n)

    # Build run lengths: L[0..R-1], sum L[r] = n, each L[r]  1
    L <- BuildRunLengths(n, R)

    # Compute segment start indices S[0..R-1]
    S <- RunStartsFromLengths(L)

    # Sort each run segment ascending
    SortEachSegmentAscending(P, S, L)

    # Compute metadata (start, len, min, max) for each segment
    Seg <- ComputeSegmentsMeta(P, S, L)

    # Reorder segments by strictly decreasing maximum value
    OrderByMaxDescending(Seg)

    # Concatenate segments in the new order
    A <- ConcatBySegments(P, Seg)

    return A
```

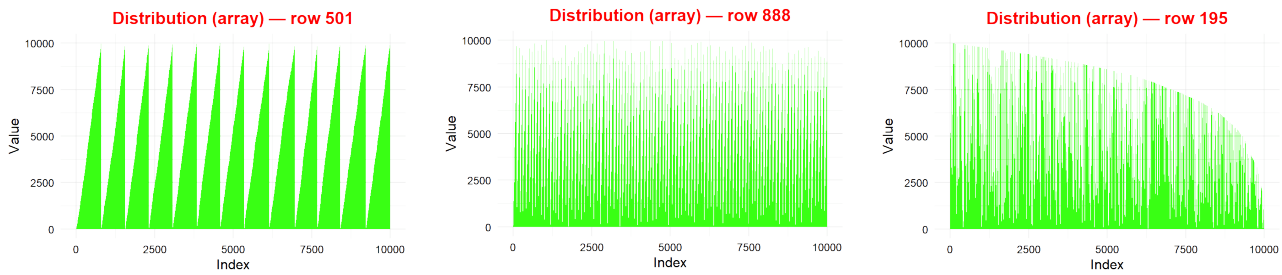
Complexity. Sorting costs $\sum_r O(L_r \log L_r) = O(N \log(N/R))$ in balanced cases; concatenation $O(N)$. Space $O(R)$ for metadata plus $O(N)$ for output.

Correctness guarantees. Claim 1 (length and range). A permutation of $\{1, \ldots, N\}$ is built, then partitioned into R positive-length segments whose lengths sum to N. Local reordering and

final concatenation are permutations of the same multiset; hence $|A| = N$ and the value range stays $\{1, \ldots, N\}$.

Claim 2 (strict increase within segments). Each segment is sorted in ascending order in-place before any global reordering, so within every segment the final output is strictly increasing.

Claim 3 (strict drops at boundaries and run count). For every segment its minimum and maximum are recorded (after local sort). The segments are then ordered by *strictly decreasing* maxima. Let two consecutive segments in this order have maxima $M_1 > M_2$. The last value of the first segment equals M_1, while the first value of the second segment is at most M_2. Therefore the concatenation point satisfies $M_1 > M_2 \geq \min(\text{next segment})$, i.e., a strict drop occurs. Combining Claims 2 and 3, each boundary contributes exactly one run break and no other breaks exist; hence the output contains exactly R strictly increasing runs. $\square$



(a) $R = \lceil \log_2 N \rceil$ **(b)** $R = \lfloor \sqrt{N} \rfloor$ **(c)** $R = N/4$

Figure 2: Permutation–segmentation method ($N = 10{,}000$): evolution of the index–value pattern as the number of runs R increases.

4.5.4 Distribution and order of elements

Figure 2 illustrates how the index–value pattern evolves under the permutation–segmentation construction. The array is assembled from R strictly increasing runs, each formed from a contiguous segment of the sorted permutation; runs are then concatenated in *decreasing order of their maxima*. As a result, every run traces an increasing slope, and each run boundary produces a sharp vertical drop to a lower starting value. The visual shape of the plot depends on the balance between run length ($\approx N/R$) and the ordering of run maxima. This yields the following characteristic structure:

- **Within-run ramps.** Inside each run the values increase strictly, forming a clean ramp in the plot. Concatenating such ramps produces a repeating "sawtooth" pattern: each run ascends, then a boundary causes a strict drop to the start of the next run. This is especially clear in the first plot with $R = \lceil \log_2 N \rceil$, where runs are long and the ramps are visually prominent.
- **Monotonically decreasing peaks and curved upper envelope.** Runs are ordered by *strictly decreasing* run maxima, so the peak of each "tooth" decreases from left to right, yielding a monotone sequence of peaks (an upper envelope). As R increases, typical run lengths $L \approx N/R$ shrink, pushing peaks downward; sorting runs by these maxima then produces a smoothly *curved* decreasing envelope. This progression is visible across the three plots: with $R = \lceil \log_2 N \rceil$ the peaks sit near N; with $R = \lfloor \sqrt{N} \rfloor$ the peaks remain high but denser; with $R = N/4$ the peaks are much lower and the envelope becomes a smooth, downward–curving arc due to very short runs.

4.5.5 Limitations

Although the method produces arrays with exactly R strictly increasing runs and allows direct control over run count, the structure it generates is not uniform over all such arrays. Several systematic biases arise from the construction:

- **Run–length distribution is multinomial, not uniform over compositions.** The run lengths $L_1, \ldots, L_R$ are distributed as

  $$(L_1, \ldots, L_R) \overset{d}{=} 1 + \text{Multinomial}\big(N - R; \tfrac{1}{R}, \ldots, \tfrac{1}{R}\big),$$

  i.e., one element per run plus a multinomial allocation of the remaining $N - R$ positions. This favors more balanced run lengths compared with a distribution that is uniform over all positive compositions of N into R parts.

- **Global ordering by maxima imposes a monotone envelope.** Ordering runs by strictly decreasing maxima enforces a globally decreasing sequence of run peaks. This guarantees clear boundaries but excludes arrangements that would produce non-monotone peak sequences.

- **No ties within or across runs.** Values form a permutation of $\{1, \ldots, N\}$, and within each run the order is strictly increasing. Consequently, duplicates cannot occur anywhere (neither within a run nor across runs), which may limit representativeness for workloads that contain equal elements or non-decreasing runs.

- **No decreasing runs.** Each segment is sorted in strictly ascending order and segments are concatenated without reversal. Therefore, runs are always increasing; arrays containing monotone *decreasing* segments cannot be generated under this construction.

4.6 Run-structured arrays (sorted-base–segmentation method)

4.6.1 Model

In contrast to the previous method, the construction begins from an already sorted sequence $[1, 2, \ldots, N]$. The index range is then partitioned into R contiguous, positive-length segments whose lengths sum to N, and the resulting segments are reordered by strictly *descending* segment maxima before concatenation. The local sorting step from the permutation–segmentation procedure is unnecessary in this case, as the base array is already strictly increasing.

4.6.2 Segment construction and ordering.

Exactly as in the previous method: use a positive-length partition of $[1..N]$ into R contiguous segments and reorder these segments by strictly decreasing segment maximum; concatenation in this order makes every boundary a strict drop while each segment stays strictly increasing inside, yielding exactly R runs.

4.6.3 Algorithmic realization (pseudocode)

```
procedure GenerateRunsFromSortedSegments(n, R):
```

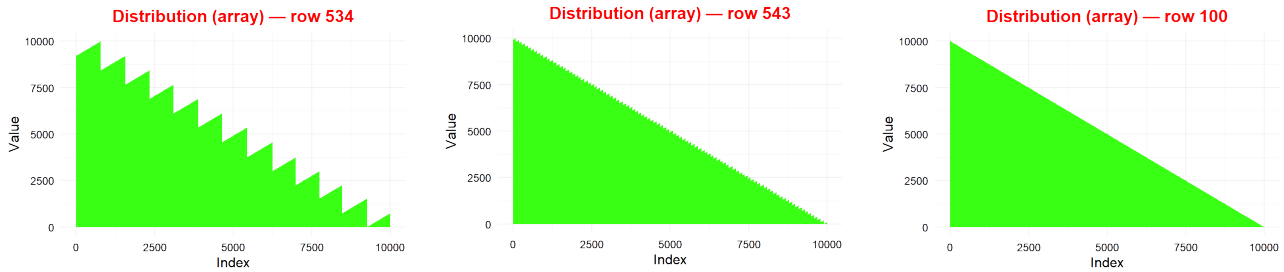
```
# Preconditions:
#   1 <= R <= n

# Strictly increasing base array 1..n
A <- new array of length n
for i <- 0 to n-1 do
    A[i] <- i + 1
end for

# Positive run lengths summing to n
L <- BuildRunLengths(n, R)

# Convert lengths to start indices
S <- RunStartsFromLengths(L)

# Compute per-run metadata (start, len, min, max)
Seg <- ComputeSegmentsMeta(A, S, L)

# Order runs so that run maxima form a strictly decreasing sequence
OrderByMaxDescending(Seg)

# Concatenate segments
return ConcatBySegments(A, Seg)
```

Complexity. No local sorts; $O(N)$ time for base + metadata + concatenation. Space $O(R)$ for descriptors and $O(N)$ for output.

Correctness guarantees. Claim 1 (length and range). The base is the strictly increasing sequence $[1, \ldots, N]$. Segment lengths are positive and sum to N; concatenation reorders contiguous slices without duplication or loss. Thus $|A| = N$ and all values remain in $\{1, \ldots, N\}$.

Claim 2 (strict increase within segments). Every segment is a contiguous slice of the sorted base and is therefore strictly increasing without additional work.

Claim 3 (strict drops at boundaries and run count). Segments are ordered by *strictly decreasing* segment maxima. At each boundary, the last value of the left segment exceeds the first value of the right segment, creating a strict drop. Hence exactly one run boundary appears per adjacent segment pair and the array has precisely R strictly increasing runs. □



(a) $R = \lceil \log_2 N \rceil$ **(b)** $R = \lfloor \sqrt{N} \rfloor$ **(c)** $R = N/4$

Figure 3: Sorted–base–segmentation method ($N = 10{,}000$): index–value distribution for increasing R.

4.6.4 Distribution and order of elements

The sorted–base segmentation model begins from a globally sorted array and partitions it into R contiguous blocks. These blocks are then concatenated in the order of *strictly decreasing maximum value*. As seen in Figure 2, each block forms an increasing ramp, and the reordering of blocks induces a descending sequence of peak values across the array. This produces the following visible structure:

- **Inside each run (ramp).** Inside each run the segment is already sorted (it is a contiguous interval of the base), so values increase with unit increment. Concatenating these ramps yields a repeating "sawtooth": a linear rise up to the run maximum followed by a strict drop at the boundary. With $R \approx \log_2 n$ the teeth are long and clearly separated; with $R \approx \sqrt{n}$ they are shorter and more frequent; with $R = n/4$ they are numerous and densely packed.
- **Nearly linear decreasing upper envelope.** Each run is a contiguous block of base values, hence its maximum is the right endpoint of that block. Ordering runs by strictly decreasing maxima makes the sequence of run peaks globally decreasing. Because endpoints from disjoint contiguous blocks step down by roughly comparable amounts when R is large, the upper envelope appears close to a straight, gently descending line. Across the three plots ($R = \lceil \log_2 n \rceil$, $R = \lfloor \sqrt{n} \rfloor$, $R = n/4$) this evolution is visible: from coarse, high steps to a finely toothed, almost linear diagonal as R increases.

4.6.5 Limitations

Since this construction differs from the permutation–segmentation method only in starting from a sorted base rather than a permutation, the resulting structural constraints are closely aligned. Key limitations are:

- **Run-length distribution.** Run lengths are formed by giving one element to each run and distributing the remaining $N - R$ units uniformly at random over the R runs,

$$(L_1, \ldots, L_R) \overset{d}{=} 1 + \text{Multinomial}\big(N - R; \tfrac{1}{R}, \ldots, \tfrac{1}{R}\big),$$

  which is not uniform over all positive compositions of N into R parts and tends to favor balanced runs.

- **Monotone peak envelope (by design).** Global reordering by strictly decreasing run maxima enforces a decreasing sequence of peaks and excludes arrays whose run peaks would be non-monotone.
- **Contiguous value blocks and no duplicates** Every run is a contiguous value interval from $[1, \ldots, N]$; values do not interleave across runs, duplicates cannot occur, and non-decreasing runs are out of scope for this construction.

4.7 Overview

The presented generators collectively define a structured framework for constructing input arrays with varying types and degrees of disorder. Each generator controls distinct aspects of data variability, enabling systematic analysis of how presortedness metrics respond to different structural conditions.

The *random permutation* model represents the reference case of complete disorder: all elements are unique, and every ordering is equally probable. It establishes the baseline level of metric variability in the absence of any imposed structure. The *k–value pool* model adds controlled duplication by limiting the number of available distinct values while preserving independence between positions. This design isolates the influence of repeated elements on metrics sensitive to value diversity, such as the number of distinct elements, distinct entropy, or value–frequency entropy. The *multiset–permutation* model further refines this idea by fixing the number of distinct labels exactly and randomizing only the frequency composition and permutation of the resulting multiset. Finally, the three *run–structured* generators introduce explicit positional order through a prescribed number of increasing runs. By modifying how run boundaries are distributed (noise–injection, permutation–segmentation, or sorted–base–segmentation), these constructions make it possible to study how run density and arrangement affect order–based metrics such as runs, run entropy, inversions, oscillations, and displacement. At the same time, these models tend to produce run lengths concentrated around N/R, which results in a somewhat repetitive, sawtooth-like global structure; while this facilitates controlled experimentation, it does not cover highly skewed or irregular run-length profiles.

All models share several desirable properties: they are parameterized by clearly interpretable structural quantities (e.g., number of runs, distinct values, or pool size), have well-defined output ranges, and can be generated in linear or near-linear time. These characteristics make them suitable for large-scale empirical studies where each structural factor can be varied independently.

In summary, the set of generators provides a consistent methodological basis for evaluating presortedness metrics across a spectrum of controlled conditions—from fully random to partially ordered arrays, and from unique to duplicate-heavy value distributions.

5 Sampling Methodology

This section formalizes the sampling strategies used to extract representative subsets of indices from input arrays under bounded resource constraints. The objective is to design sampling procedures that (i) preserve the original order of elements, (ii) balance local continuity with global coverage, and (iii) maintain fairness across different regions of the array. Each approach defines a rule for selecting indices based on a fixed sampling budget m, controlling how densely and where within the array elements are chosen while ensuring that their relative order is preserved. These strategies enable systematic analysis of how different sampling configurations influence the resulting values of presortedness metrics computed on samples.

Three complementary sampling schemes are considered:

- **Cluster-Based Sampling** selects several contiguous segments ("clusters") of indices, optionally extending them to meet the target sample size. This method emphasizes locality and structural coherence, making it suitable for metrics that capture positional organization, such as runs, run entropy, oscillations or removals.
- **Stratified Sampling** partitions the array into consecutive strata and draws elements randomly from each stratum according to its size, providing balanced representation across the full index range. This scheme prioritizes uniform global coverage and reduces regional sampling bias.
- **Combined Sampling** integrates both principles by dividing the array into strata and selecting one contiguous block ("cluster") within each. The resulting samples preserve local order while maintaining even spatial distribution across the array.

In all methods, sampled indices are emitted in strictly increasing order to preserve the input's original ordering. Each sampling strategy operates in linear time with respect to the sample size, offering a controllable balance between locality, representativeness, and computational efficiency.

5.1 Cluster-Based Sampling

5.1.1 Model

Construct a sample of size m from an array $a[0..n-1]$ by (i) placing $h = \lfloor m/cl \rfloor$ disjoint base clusters of equal length cl inside $[0, n)$, (ii) distributing the remainder ($r = m - h \cdot cl$) by right-extending eligible clusters in unbiased rounds (no segment grows twice within the same round), and (iii) emitting all covered indices in increasing order (the sample is globally index-ordered).

Definition. A *cluster* is a half-open integer interval $I = [s, e) \subseteq [0, n)$ with length $|I| = e - s \geq 1$. A *cluster configuration* with parameters (h, cl) is an ordered family $(I_1, \ldots, I_k)$ such that

$$0 \leq s_1 < e_1 \leq s_2 < e_2 \leq \cdots \leq s_k < e_k \leq n, \qquad |I_i| = L \ \ \forall i.$$

The *coverage* is $(\sum_{i=1}^{k} |I_i| = h \cdot cl)$.

Definition. For the base clusters $C_i = [s_i, e_i)$, define the cluster right-extension capacity as

$$\ell_i^{\max} = \begin{cases} s_{i+1}, & i < h, \\ n, & i = h, \end{cases} \qquad c_i = \ell_i^{\max} - e_i \in \mathbb{Z}_{\geq 0}.$$

Cluster layout. Let $(G := n - h \cdot cl)$ be the remaining capacity after placing h length-cl clusters. Introduce nonnegative integer *gaps* $(g_0, \ldots, g_h)$ with $\sum_{i=0}^{h} g_i = G$, and define

$$s_1 = g_0, \qquad s_i = g_0 + (i-1)cl + \sum_{t=1}^{i-1} g_t \ \ (2 \leq i \leq h), \qquad e_i = s_i + cl.$$

This parameterizes all feasible base configurations. Sampling $(g_0, \ldots, g_h)$ uniformly from all solutions of $\sum g_i = G$ (classical stars-and-bars) makes every base configuration equiprobable, with

$$\#\mathcal{C}(n, h, cl) = \binom{G+h}{h} = \binom{n - h \cdot cl + h}{h}.$$

Remainder distribution. Let $(r := m - h \cdot\ cl)$. Form the eligible set

$$E \;=\; \{\, i \in \{1, \ldots, h\} : c_i > 0 \,\} \quad \text{with} \quad c_i \;=\; \ell_i^{\max} - e_i.$$

Only clusters in E can be extended without overlap. In each round, choose uniformly at random, *without replacement*, a subset of clusters from E and increase each chosen cluster by one position (apply $e_i \leftarrow e_i + 1$ and $c_i \leftarrow c_i - 1$), decreasing r accordingly.

5.1.2 Algorithmic realization (pseudocode).

```
procedure ClusterSample(a, m, L):
    # Preconditions: 1 <= L <= m <= n (= length(a))

     k <- floor(m / L); place h disjoint base clusters of length cl in [0,n)
       using a UNIFORM stars-and-bars layout.

     r <- m - h * cl; compute per-cluster right-extension capacities c_i;


     While r > 0:
         Let E = { i : c_i > 0 }.
         Choose uniformly at random, without replacement, some clusters from E
         (up to r of them); extend each chosen cluster by +1; update c_i and r.

     Return the sample as the ordered union of all covered indices
```

Complexity. Stars-and-bars sampling and base construction take $O(h \log h)$ time and $O(h)$ space; the extension loop performs $(m - h \cdot\ cl)$ unit updates with per-round eligibility scans, i.e., $O(h + m - h \cdot cl)$ expected time; emission is $O(m)$. Overall $O(h \log h + m)$ expected time and $O(h)$ auxiliary space for cluster metadata plus $O(m)$ for outputs.

Sample-size and feasibility guarantees. Let $h = \lfloor m/cl \rfloor$ and write the stars-and-bars gaps as nonnegative integers $(g_0, \ldots, g_h)$ with $\sum_{i=0}^{h} g_i = G := n - h \cdot L$. For base clusters $C_i = [s_i, e_i)$ with $e_i = s_i + cl$ and

$$s_1 = g_0, \qquad s_i = g_0 + (i-1)L + \sum_{t=1}^{i-1} g_t \;\; (2 \le i \le h),$$

the right-extension capacities are

$$c_i \;=\; \ell_i^{\max} - e_i \;=\; \begin{cases} s_{i+1} - (s_i + cl), & i < h, \\ n - (s_h + cl), & i = h, \end{cases} \quad \Rightarrow \quad c_i = \begin{cases} g_i, & i < h, \\ g_h, & i = h. \end{cases}$$

Hence the *total capacity* equals the total slack:

$$\sum_{i=1}^{h} c_i \;=\; \sum_{i=0}^{h} g_i \;=\; G \;=\; n - h \cdot L.$$

Let the remainder be $r := m - h \cdot cl$. Under the standing precondition $m \leq n$,

$$r \;=\; m - h \cdot cl \;\leq\; n - h \cdot cl \;=\; \sum_{i=1}^{h} c_i,$$

so the remainder never exceeds the aggregate capacity of the cluster set. The round-wise extension rule only applies a unit growth to clusters with $c_i > 0$, thus never violating feasibility ($e_i < \ell_i^{\max}$ is preserved). Each accepted extension reduces r by exactly 1; after exactly r unit extensions the final coverage is

$$h \cdot cl + r \;=\; h \cdot cl + (m - h \cdot cl) \;=\; m,$$

so the returned sample contains *exactly* m elements.

5.1.3 Coverage analysis (cluster length scaling).

To illustrate how cluster length controls the balance between locality and global coverage, Figures 4–6 show sample index coverage patterns for $n = 10{,}000$ and sample size $m = 300$ under three different base cluster lengths:

$$\text{(i) } cl = \lfloor \log_2 m \rfloor, \qquad \text{(ii) } cl = \lfloor 2 \log_2 m \rfloor, \qquad \text{(iii) } cl = \lfloor 2\sqrt{m} \rfloor.$$

Since the number of clusters is $h = \lfloor m/cl \rfloor$, increasing cl reduces the number of clusters while increasing their individual lengths. This parameter directly adjusts the trade-off between *local continuity* inside clusters and *global dispersion* across the array.

(1) $cl = \lfloor \log_2 m \rfloor$**.** The sample is composed of many short clusters evenly dispersed across the full index range. In the coverage plot, these clusters appear as narrow, evenly spaced vertical strips, with only small gaps between them. The result is a finely grained, nearly uniform coverage that preserves local ordering within each cluster while maintaining broad, evenly spread global coverage.

(2) $cl = \lfloor 2 \log_2 m \rfloor$**.** The number of clusters decreases, and each cluster becomes visibly wider. Coverage appears as moderately spaced vertical blocks separated by empty corridors of several hundred indices. The sample retains global dispersion but begins to emphasize local structure within each cluster—coverage looks chunkier and less continuous than in case (1).

(3) $cl = \lfloor 2\sqrt{m} \rfloor$**.** Only a few long clusters remain, each covering an extended contiguous region of the array. Between these blocks, large uncovered intervals emerge, giving the coverage a patchy, segmented appearance. This configuration maximizes local continuity inside clusters but sacrifices global evenness, creating strong locality and wide gaps.

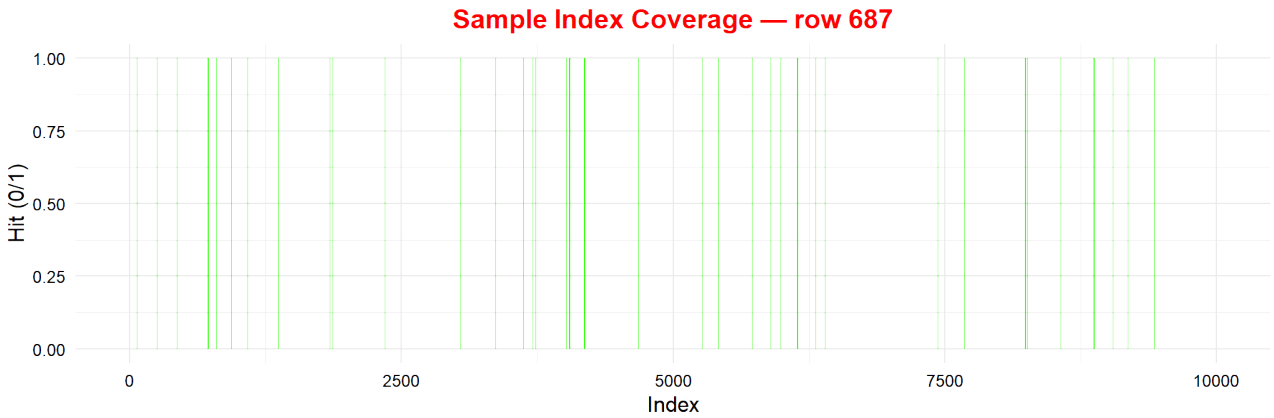


Figure 4: Sample index coverage, $n = 10{,}000$, $m = 300$, cluster-based sampling, cluster length $cl = \lfloor \log_2 m \rfloor$.

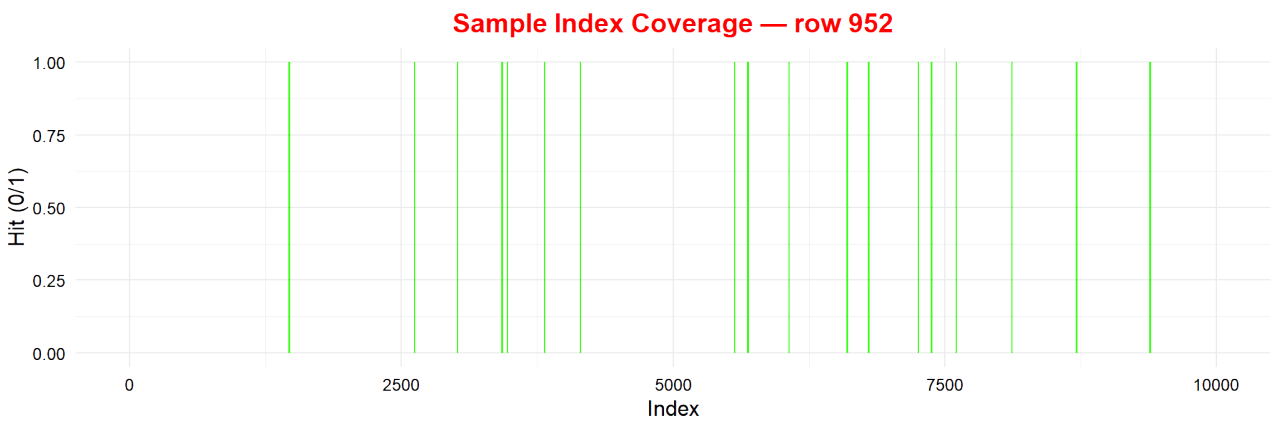


Figure 5: Sample index coverage, $n = 10{,}000$, $m = 300$, cluster-based sampling, cluster length $cl = \lfloor 2 \log_2 m \rfloor$.

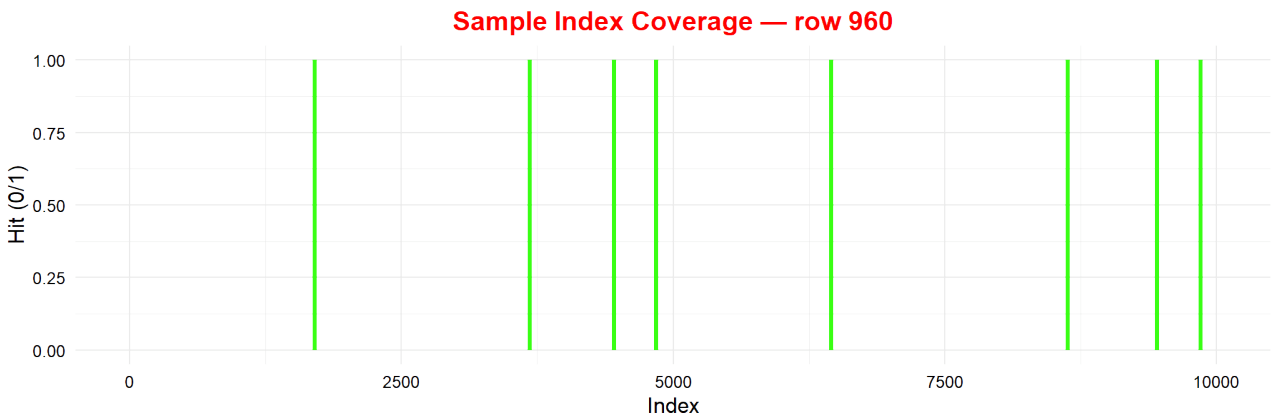


Figure 6: Sample index coverage, $n = 10{,}000$, $m = 300$, cluster-based sampling, cluster length $cl = \lfloor 2\sqrt{m} \rfloor$.

5.1.4 Limitations.

- **Sensitivity to cl.** The locality/coverage trade–off is governed almost entirely by cl: small cl achieves broad reach but shallow local views; large cl gives good local context but risks missing narrow phenomena between clusters.
- **No adaptivity to content.** Placement is oblivious to the array's structure (runs, duplicates, inversions). Clusters may land in uninformative regions while skipping informative ones.
- **Remainder fairness is approximate.** The "one–per–round" extension limits runaway growth but still depends on per–cluster capacity; early exhaustion of some capacities can shift more remainder to the remaining clusters, slightly increasing heterogeneity.

5.2 Stratified Sampling

Model. The array $a[0..n)$ is divided into h consecutive strata, each having a target length sl (the final stratum may be shorter). Every stratum receives an initial quota $q_i = 1$, ensuring that each region of the array contributes to the sample. The remaining sampling capacity is then distributed across strata in an unbiased manner, with each stratum limited by its own capacity cap_i (the maximum number of additional elements it can admit).

Definition. A *stratum* is a half-open interval $S_i = [start_i, end_i) \subseteq [0, n)$ with length $sl_i = end_i - start_i \geq 1$ and integer *quota* $q_i \in \{0, 1, \ldots, sl_i\}$. A *stratified configuration* is a family $(S_1, \ldots, S_h)$ that tiles $[0, n)$ without overlap and a quota vector $q = (q_1, \ldots, q_h)$ with $\sum_i q_i = m$.

Strata construction. With target size m, define

$$h = \left\lceil \frac{n}{m} \right\rceil, \qquad start_i = (i-1)m, \qquad end_i = \min\{i * m,\, n\}, \quad i = 1, \ldots, h,$$

so that $sl_i = sl$ for $i < h$ and $sl_h \in [1, sl]$. These strata are contiguous, disjoint, and cover $[0, n)$.

Quota allocation (base + unbiased extras). Initialize the *base* quotas $q_i \leftarrow 1$ for all i (requires $m \geq h$). Let the per-stratum *capacity* after the base be

$$\text{cap}_i = \max(0, \ell_i - 1),$$

the maximum extra units admissible in stratum i. Let the leftover quota be extra $= m - h$. While extra > 0: collect the *active* set $A = \{\, i : \text{cap}_i > 0 \,\}$; sample uniformly at random, *without replacement*, a subset of A; give $+1$ to each chosen stratum (decrementing cap_i accordingly); decrease extra. Each round assigns at most one increment per active stratum ("unique per round"), which removes left-to-right bias.

Within-stratum selection. For each stratum i, if $q_i = sl_i$ take all indices in $[start_i, end_i)$; otherwise draw a uniform q_i-subset of $\{0, \ldots, \ell_i - 1\}$ (e.g., Floyd's method), shift by $start_i$, and sort locally. Concatenating per-stratum lists in increasing stratum order yields a globally sorted index set of size m.

5.2.1 Algorithmic realization (pseudocode)

```
procedure StratifiedSample(a, m, sl):
    # Preconditions:
    n = length(a), sl > 0, 1 <= m <= n, k = ceil(n/sl) <= m.

    Build consecutive strata of target size sl (last may be shorter)

     Let h = number of strata; require m >= h.
       Initialize quotas: q_i <= 1 for all i.
       Let cap_i <= max(0, len_i - 1) and extra <= m - h.

     While extra > 0:
         Let A = { i : cap_i > 0 }.
         Choose uniformly at random, without replacement, some strata from A
         (up to extra of them); for each chosen i: q_i <- q_i + 1; cap_i <- cap_i  1;
         extra <- extra  1 per assignment.

     For each stratum i:
         If q_i = len_i: take all indices in [start_i, end_i);
         else draw q_i unique indices uniformly inside the stratum.

     Return values and indices concatenated in stratum order.
```

Complexity. Strata construction is $O(h)$. Quota allocation is $O(R * h)$ in the worst case with R rounds (typically small). Within-stratum selection totals $O(m)$ expected time using incremental unique sampling; assembling the sample is $O(m)$. Space: $O(h)$ for stratum metadata and $O(m)$ for outputs.

Sample-size and feasibility guarantees. To guarantee at least one sampled element from every stratum, there must exist quotas q_i with $1 \le q_i \le \ell_i$ and $\sum_i q_i = m$. A necessary and sufficient condition is

$$1 \le h \le \min\{m, n\} \qquad \text{and} \qquad h \le m \le n.$$

Under these inequalities one can choose $q_i \ge 1$ for all i and distribute the remaining extra $:= m - h$ units within the per-stratum capacities. Define the extra capacity of stratum i as

$$\text{cap}_i := sl_i - 1 \ \ (\ge 0),$$

so that the aggregate extra capacity is

$$\sum_{i=1}^{h} \text{cap}_i = \sum_{i=1}^{h} (sl_i - 1) = \Big(\sum_{i=1}^{h} sl_i\Big) - h = n - h.$$

Since $m \le n$, it follows that

$$\sum_{i=1}^{h} \text{cap}_i = n - h \ge m - h = \text{extra}.$$

Consequently, each stratum contributes at least one element ($q_i \geq 1\ \forall i$), and the quotas sum to m; thus the stratified sampler selects *at least one* index from every stratum and returns a sample of the *required size* m.

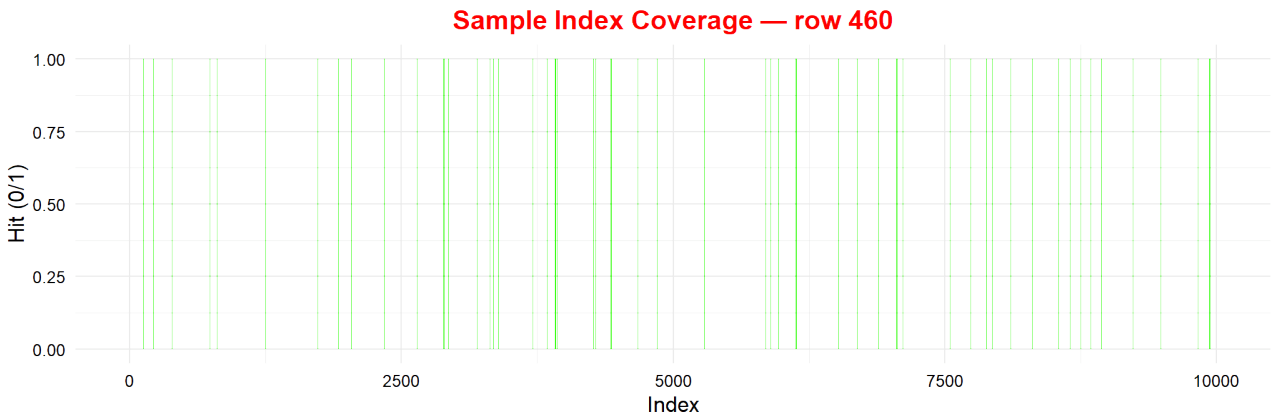


Figure 7: Sample index coverage, $n = 10{,}000$, $m = 300$, stratified sampling, stratum length $sl = 300$.

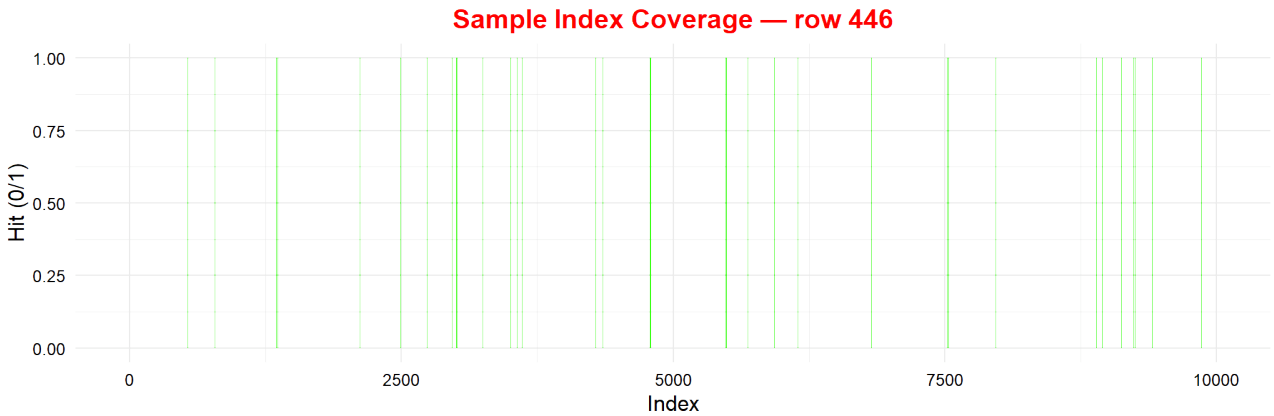


Figure 8: Sample index coverage, $n = 10{,}000$, $m = 300$, stratified sampling, stratum length $sl = n/10 = 1{,}000$.

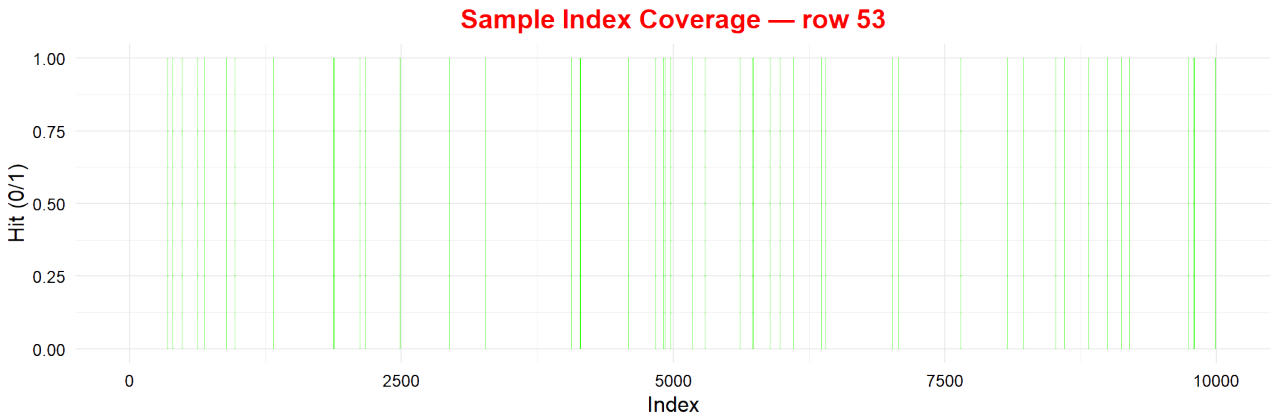


Figure 9: Sample index coverage, $n = 10{,}000$, $m = 300$, stratified sampling, stratum length $sl = n/4 = 2{,}500$.

5.3 Coverage analysis (stratum-length scaling).

The effect of stratum size on index coverage is illustrated in Figures 7–9. For $n = 10{,}000$ and sample size $m = 300$, three stratum-length schedules are examined:

$$\text{(i) } sl = m = 300, \qquad \text{(ii) } sl = n/10 = 1{,}000, \qquad \text{(iii) } sl = n/4 = 2{,}500.$$

Varying sl controls the balance between global dispersion (many small strata) and local concentration (few large strata). Smaller strata enforce more uniform coverage across the array, while larger strata create wider contiguous regions within which sampling is free to vary.

(1) $sl = m = 300$**.** The line is partitioned into many narrow bands. Within each band, hits appear as thin, fairly even ticks; band borders are subtle, and long empty stretches are absent. The overall impression is a fine, almost continuous pattern along the axis.

(2) $sl = n/10$**.** Bands become wider and fewer. Within each wide band, ticks form small, dense groups; between bands, clean corridors with no hits are visible. Global spread remains balanced, while the banded structure becomes visually pronounced.

(3) $sl = n/4$**.** Only a handful of very wide bands remain. Each band is densely populated with hits, and the gaps between bands are long and visually quiet. A coarse periodic pattern (occupied panel, empty corridor, occupied panel, ...) dominates, and the fine intra-band spacing is less salient.

5.3.1 Limitations.

- **Over-fragmentation of context.** If sl is small, local patterns that span more than one stratum—or sit on stratum borders—can be split and partly missed.
- **Within-stratum locality loss.** Because individual indices are sampled inside each stratum (not a contiguous block), continuous patterns inside a stratum can be missed.
- **No adaptivity to content.** The scheme guarantees at least one element per stratum but does not adapt quotas to content (e.g., run density or duplicates). Regions with more "signal" are treated the same as quiet regions.

5.4 Combined Sampling (stratified–clustered)

5.4.1 Model

Partition $[0, n)$ into $h = \lceil n/sl \rceil$ consecutive, non-overlapping strata of target size $sl > 0$ (the last may be shorter). Allocate an integer *quota* q_i to each stratum so that every stratum contributes at least one element and $\sum_i q_i = m$. Inside each stratum i, select *exactly one contiguous cluster* of length q_i (if $q_i = sl_i$, take the whole stratum; otherwise choose a start offset uniformly from $\{0, \ldots, sl_i - q_i\}$). Concatenate per-stratum clusters in stratum order; the resulting index list is globally increasing and has length m. All definitions, strata construction, and quota allocation from the *Stratified Sampling* subsection carry over unchanged.

Within-stratum cluster selection. For each stratum i with quota q_i:

$$C_i \;=\; \begin{cases} [start_i, end_i), & \text{if } q_i = \ell_i, \\ [start_i + U_i,\ start_i + U_i + q_i), & \text{if } q_i < \ell_i, \end{cases}$$

where $U_i \sim \mathrm{Unif}\{0, \ldots, sl_i - q_i\}$ independently across strata. Indices inside C_i are emitted in increasing order.

5.4.2 Algorithmic realization (pseudocode).

```
procedure CombinedSample(a, m, sl):
    # Preconditions: n = length(a), sl > 0, 1 <= m <= n, k = ceil(n/sl) <= m

    Build consecutive strata of target size sl (last may be shorter)

    Let h = number of strata; require m >= h.
       Initialize quotas: q_i <= 1 for all i.
       Let cap_i <= max(0, sl_i - 1) and extra <= m - h.

    While extra > 0:
         Let A = { i : cap_i > 0 }.
         Choose uniformly at random, without replacement, some strata from A
         (up to extra of them); for each chosen i: q_i <- q_i + 1; cap_i <- cap_i  1;
         extra <- extra  1 per assignment.

    For each stratum i:
           if q_i = len_i: cluster C_i = [start_i, end_i);
           else draw U_i  {0..len_i  q_i} uniformly and set
                C_i = [start_i + U_i, start_i + U_i + q_i).

    Return values/indices by concatenating all C_i in stratum order.
```

Complexity. Strata construction is $O(h)$. Quota allocation costs $O(R * h)$ in the worst case over R rounds (typically small). Cluster selection is $O(1)$ per stratum plus $O(q_i)$ to emit indices, totaling $O(m)$. Space is $O(h)$ for stratum metadata and $O(m)$ for outputs.

Sample-size and feasibility guarantees. Identical to the proof in *Stratified Sampling*. In particular, feasibility requires $1 \leq h \leq \min\{m, n\}$ and $h \leq m \leq n$; the aggregate extra capacity accommodates extra $= m - h$; hence each stratum contributes at least one element and the concatenated clusters contain exactly m indices in globally increasing order.

5.4.3 Coverage analysis (stratum-length scaling).

In the combined approach, each stratum contributes a single contiguous block of indices of length q_i. As a consequence, the coverage pattern naturally arranges into a sequence of bands: one band per stratum, each containing a solid rather than scattered points. The spacing between bands is determined by the chosen stratum length sl, while the thickness of each block reflects how the sample budget is distributed across strata. Varying sl thus shifts the balance between local continuity within each block and global dispersion across the array, as shown in Figures 10–12.

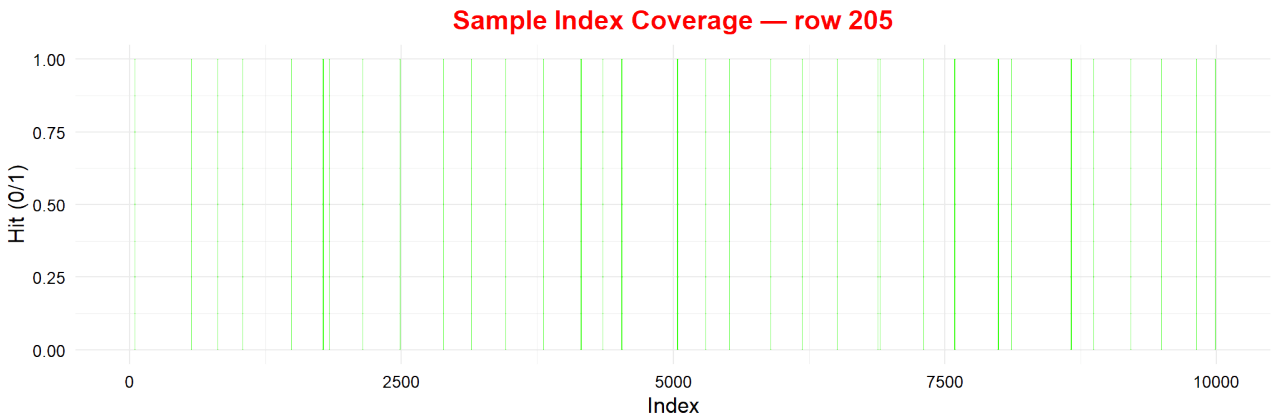


Figure 10: Sample index coverage, $n = 10{,}000$, $m = 300$, combined (stratified–clustered) sampling, stratum length $sl = m = 300$.

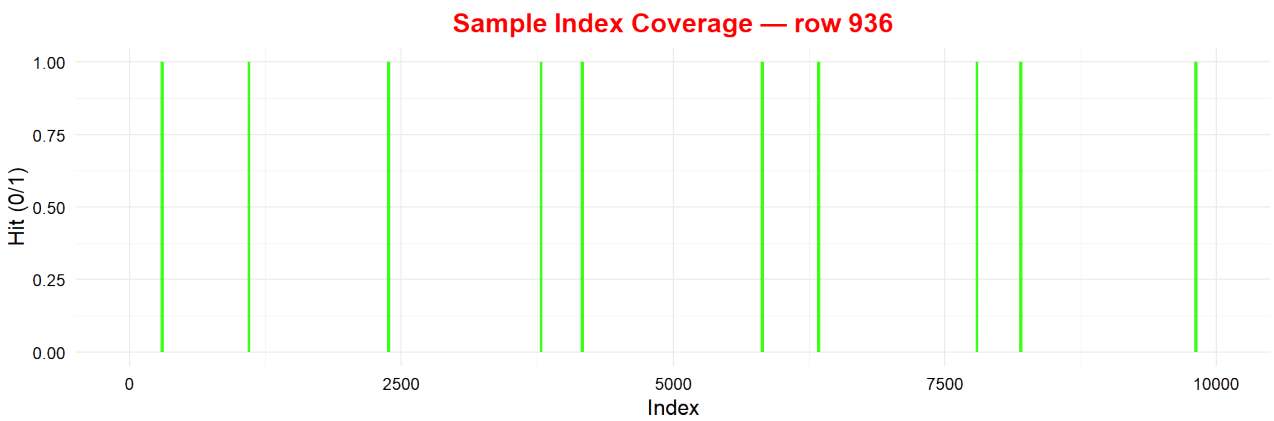


Figure 11: Sample index coverage, $n = 10{,}000$, $m = 300$, combined (stratified–clustered) sampling, stratum length $sl = n/10 = 1{,}000$.

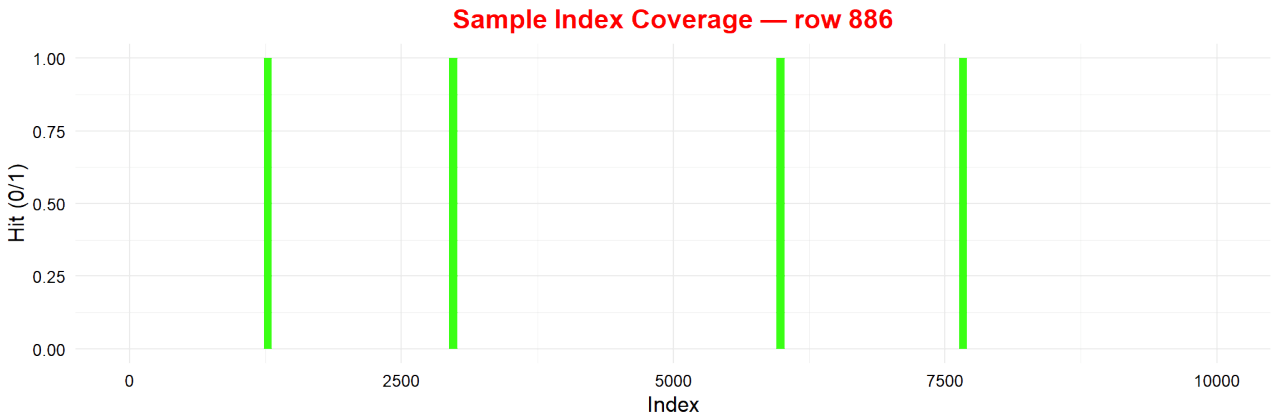


Figure 12: Sample index coverage, $n = 10{,}000$, $m = 300$, combined (stratified–clustered) sampling, stratum length $sl = n/4 = 2{,}500$.

(1) $sl = m = 300$**.** The line is partitioned into many narrow bands ($h \approx 34$). Inside every band a short contiguous block of ~ 9 indices is visible, producing fine, regularly spaced slivers across the axis. Empty corridors between blocks are present but short; overall dispersion looks uniform at the global scale while clearly clustered locally within each band.

(2) $sl = n/10$**.** Bands become wider and fewer ($k = 10$). Within each wide band the block thickens to a run of ~ 30 consecutive hits. Between adjacent bands long clean corridors with no hits appear, making the banded, periodic structure visually prominent. Local context inside a band is strong; coverage between bands is intentionally sparse.

(3) $sl = n/4$**.** Only a handful of very wide bands remain ($h = 4$). Each band contains a long solid block (~ 75 consecutive indices), and the gaps between bands become extended, visually quiet stretches. .

5.4.4 Limitations.

- **Blockiness inside strata.** One contiguous block per stratum preserves local order but cannot capture multiple disjoint phenomena within the same stratum.
- **Corridor artifacts for large** sl**.** As sl grows, uncovered corridors between neighboring blocks widen and the coverage takes on a strong periodic pattern of populated bands separated by long empty regions, reducing global uniformity.
- **No adaptivity to content.** Start offsets are chosen uniformly within strata; block placement does not adapt to the array's structure, so informative subregions can be missed while uninformative ones are overrepresented.

5.5 Overview

Across all three samplers the common goal is to select m indices in strictly increasing order while balancing global reach against local context. The methods offer complementary points on this trade–off:

- **Cluster-Based Sampling** maximizes locality by emitting a small number of contiguous blocks whose sizes are controlled by cl. Stars–and–bars gives good global dispersion on average, but

large uncovered corridors may appear when $h = \lfloor m/cl \rfloor$ is small. Remainder rounds preserve fairness and avoid positional bias.

- **Stratified Sampling** maximizes *fairness across regions*: every stratum contributes, and per–stratum picks are uniformly distributed. This yields excellent global coverage but only pointwise views locally (inside each stratum the indices are scattered rather than contiguous).
- **Combined (Stratified–Clustered)** preserves the per–stratum fairness of stratification while restoring locality inside each stratum via a single contiguous block of length q_i. The result is banded coverage with solid blocks and empty corridors between bands when strata are wide.

All three implementations run in $O(m)$ output time plus $O(h)$ metadata, with small overheads for layout/allocation: Clustered adds $O(h \log h)$ for stars–and–bars layout; Stratified/Combined add $O(R \cdot h)$ for a few unbiased quota rounds. Auxiliary space is $O(h)$ (metadata) + $O(m)$ (outputs).

None of the schemes is content–adaptive: placements and quotas ignore the actual structure (runs/duplicates). Large strata create visible corridors; very small strata fragment local context. These effects are intentional knobs rather than bugs and will be accounted for in the evaluation.

Taken together, the three samplers provide a controlled and complementary toolkit for building order–preserving samples: Clustered emphasizes local continuity, Stratified enforces global fairness, and Combined strikes a tunable middle ground. This suite underpins the subsequent experiments that relate sample–based presortedness estimates to full–array ground truth and stress–test adaptive sorting decisions under varying locality and coverage profiles.

6 Experiment Design

6.1 Aim and Scope

The objective of the experimental design is to evaluate whether *presortedness measures*—functions that quantify the degree of order or disorder within an array—can be reliably estimated from small, index–ordered samples while maintaining predictive power for the global structure. This question is central to determining whether a sorting system can *select an appropriate adaptive strategy* based on sample-derived information, rather than evaluating the entire input.

To explore this systematically, the experiment constructs a controlled, reproducible environment where both the generation of disorder and the sampling of elements are explicitly parameterized. Each experimental factor—array family, sample size, and sampling configuration—is designed to vary along interpretable axes, enabling the isolation of specific effects on metric behavior. The goal is to identify when sample-based metric values meaningfully track their full-array structure.

The study combines two complementary design principles. On one hand, the use of synthetic data families with analytically tractable properties ensures that changes in presortedness can be attributed to known structural manipulations such as duplication, multiplicity imbalance, or the number of ascending runs. On the other, the inclusion of several distinct sampling methods—each balancing locality and dispersion differently—makes it possible to assess how sampling geometry interacts with those same structures. Together, these dimensions form a factorial grid that spans both value–space and position–space heterogeneity.

The overall workflow is divided into three coordinated stages: *(i) data generation*, *(ii) sampling*, and *(iii) result analysis*.

- In the **generation stage**, controlled input families are produced, each with explicit parameters governing duplication, distinctness, and run count.
- In the **sampling stage**, each full array is reduced to ordered index subsets according to several sampling strategies, each with its own locality–coverage trade–off.
- In the **analysis stage**, statistical concordance between sample-based and full-array presortedness measures is quantified using correlation, bias, and variance diagnostics.

This section specifies in detail the first two stages—generation and sampling—and outlines how the corresponding datasets are organized for subsequent quantitative analysis. The evaluation and interpretation of results are presented in the next section, where metric–wise trends and sampling effects are compared across data families and parameter regimes. The full implementation of the data generators, sampling strategies, metric evaluators, and the complete experimental pipeline used in this thesis is available in the accompanying open-source repository: `https://github.com/ZeynVlado/Estimators-for-Measures-of-Presortedness`

6.2 Data Generation Configurations

6.2.1 Random Permutation (baseline)

Configuration $G_c^{\mathbf{perm}}$.

$$G_c^{\text{perm}} = \{\texttt{size} = 10{,}000\}.$$

6.2.2 Random Array from a k-Value Pool (duplicate-controlled i.i.d.)

Configuration $G_c^{k\text{-}\mathbf{pool}}$.

$$G_c^{k\text{-pool}} = \{\texttt{size} = 10{,}000,\ \texttt{min} = 1,\ \texttt{max} = 10{,}000,\ k \in \{2500, 5000, 7500, 9000\}\}.$$

Varying k modulates the expected duplicate density while maintaining uniform randomness in index space. This allows for the isolation of how metric behavior changes as value diversity decreases, independently of positional order. Long monotone runs remain improbable, so any changes in presortedness are attributable to duplication alone. The factor levels correspond to 25%, 50%, 75%, and 90% of the value universe. This monotone grid balances coverage and contrast: very small k would saturate the array with duplicates, while large k approximates the permutation baseline.

6.2.3 Multiset Permutation (exactly u distinct values)

Configuration $G_c^{\mathbf{mset}}$.

$$G_c^{\text{mset}} = \{\texttt{size} = 10{,}000,\ \texttt{base} = 1,\ u \in \{2500, 5000, 7500, 9000\}\}.$$

The same factor grid as for the k-pool family is used to align the range of #distinct values, enabling direct cross-family comparisons under equivalent distinctness levels.

6.2.4 Run-Structured Arrays (exactly R strictly increasing runs)

Configuration G_c (shared by all three methods).

$$G_c^{\text{run}} = \Big\{\texttt{size} = 10{,}000,\ R \in \Big\{\lceil \log_2 n \rceil,\ \lfloor \sqrt{n} \rfloor,\ 2\lfloor \sqrt{n} \rfloor,\ \tfrac{n}{20},\ \tfrac{n}{10},\ \tfrac{n}{4},\ \tfrac{n}{2.5}\Big\},\ \texttt{min} = 1,\ \texttt{max} = 10{,}000\Big\}.$$

These levels range from nearly sorted sequences to highly fragmented order and enable consistent comparison of run-sensitive metrics across all configurations of sampling methods. For $n = 10{,}000$, this corresponds to $R \in \{14, 100, 200, 500, 1000, 2500, 4000\}$, covering transitions from minimal to extensive disruption of ascending structure.

Table 2: Data Generation configurations G_c (all families).

Family	Symbol	Levels / Description
Random permutation	G_c^{perm}	`size`.
k-value pool (i.i.d.)	$G_c^{k\text{-pool}}$	`size`, `min`, `max`, $k \in \{2500, 5000, 7500, 9000\}$.
Multiset permutation	G_c^{mset}	`size`, `base`, $u \in \{2500, 5000, 7500, 9000\}$.
Run-structured (shared)	G_c^{run}	`size`, $R \in \{14, 100, 200, 500, 1000, 2500, 4000\}$.

6.3 Sampling Methods and Configurations

Common sample-length levels. Across all sampling methods, the target sample length m takes the same fixed levels and is held constant within each configuration while the method-specific parameter varies over its grid:

$$m \in \{100,\ 200,\ 300,\ 400,\ 500,\ 600\}.$$

This ensures that all methods operate under comparable sampling budgets, making differences in performance attributable to their locality behavior rather than to variations in sample size.

6.3.1 Cluster-Based Sampling

Configuration S_c^{cluster}. For each fixed sample length m (from the common levels above), the base cluster length cl varies over

$$cl \in \big\{\lfloor \log_2 m \rfloor,\ 2\lfloor \log_2 m \rfloor,\ \lfloor \sqrt{m} \rfloor,\ 2\lfloor \sqrt{m} \rfloor,\ m\big\}.$$

This grid spans locality regimes from very short clusters (logarithmic in m) up to one full cluster covering the entire sample ($cl = m$), while preserving feasibility.

$$S_c^{\text{cluster}} = \Big\{\ \texttt{size} = m,\ cl \in \{\lfloor \log_2 m \rfloor,\ 2\lfloor \log_2 m \rfloor,\ \lfloor \sqrt{m} \rfloor,\ 2\lfloor \sqrt{m} \rfloor,\ m\}\ \Big|\ m \in \{100, 200, 300, 400, 500, 600\}\ \Big\}.$$

Table 3: Cluster-based sampling: parameter grid S_c^{cluster}. For each m, the cluster-length set is $\{\lfloor \log_2 m \rfloor,\ 2\lfloor \log_2 m \rfloor,\ \lfloor \sqrt{m} \rfloor,\ 2\lfloor \sqrt{m} \rfloor,\ m\}$.

m	$\lfloor \log_2 m \rfloor$	$2\lfloor \log_2 m \rfloor$	$\lfloor \sqrt{m} \rfloor$	$2\lfloor \sqrt{m} \rfloor$	m
100	6	12	10	20	100
200	7	14	14	28	200
300	8	16	17	34	300
400	8	16	20	40	400
500	8	16	22	44	500
600	9	18	24	48	600

6.3.2 Strat-Based Sampling

Shared Configuration for Stratified and Combined Sampling: $S_c^{\textbf{strat/comb}}$. For both designs we use the same parameter grid. Given a fixed sample length s (from the common levels),

$$sl \in \{\, m,\ 2m,\ \lfloor n/20 \rfloor,\ \lfloor n/10 \rfloor,\ \lfloor n/4 \rfloor \},$$

where sl denotes the stratum length. The first two levels tie stratum scale directly to the global budget, while the coarser tilings $\{n/20, n/10, n/4\}$ probe robustness with respect to stratum granularity.

$$S_c^{\text{strat/comb}} = \Big\{ \texttt{size} = m,\ \ sl \in \{\, m,\ 2m,\ \lfloor n/20 \rfloor,\ \lfloor n/10 \rfloor,\ \lfloor n/4 \rfloor\} \ \Big|\ m \in \{100, 200, 300, 400, 500, 600\} \Big\}.$$

Table 4: Stratified and combined sampling: shared grid $S_c^{\text{strat/comb}}$. Columns list the concrete stratum lengths for each s (with $n = 10{,}000$).

m	$sl = m$	$sl = 2m$	$sl = \lfloor n/20 \rfloor$	$sl = \lfloor n/10 \rfloor$	$sl = \lfloor n/4 \rfloor$
100	100	200	500	1000	2500
200	200	400	500	1000	2500
300	300	600	500	1000	2500
400	400	800	500	1000	2500
500	500	1000	500	1000	2500
600	600	1200	500	1000	2500
					end of table

6.4 Experiment Organization and Data Layout

This subsection describes the concrete execution protocol that produces the datasets used in the evaluation. The experiment proceeds in two stages: (i) construction of full arrays according to the generation configurations, and (ii) extraction of ordered index-samples according to the sampling configurations. The outputs of these stages are then grouped into two aggregate datasets, *random-combined* and *run-combined*, which serve as primary analysis units.

6.4.1 Stage 1: Full-Array Generation

For every data-generation configuration G_c (Table 2), a dataset of

$$N_{\text{gen}} = 1000$$

independent arrays of length $n = 10{,}000$ is produced.

Table 5: Full-array datasets produced during Stage 1.

Family	# Configurations	Arrays per Configuration	Total Arrays
Random permutation	1	1000	1,000
k-value pool	4	1000	4,000
Multiset permutation	4	1000	4,000
Run-structured (all mechanisms, all R)	21	1000	21,000
Total	**30**	–	**30,000**

Thus, Stage 1 yields 30,000 full arrays in total.

6.4.2 Stage 2: Sampling from Full Arrays

For each full array generated in Stage 1, and for each target sample length

$$m \in \{100, 200, 300, 400, 500, 600\},$$

samples are produced using all three sampling methods (cluster-based, stratified, combined), and for each method, all parameter configurations specified in its grid. Since each method has five parameter variants per m, each full array produces:

$$6 \text{ (sample sizes)} \times 3 \text{ (methods)} \times 5 \text{ (variants)} = 90 \text{ samples.}$$

Table 6: Sampling output volume (Stage 2). Each full array yields 90 samples (6 sizes $\times$ 3 methods $\times$ 5 variants).

Sampling Method	Samples per Full Array	Total Samples
Cluster-based	30	900,000
Stratified	30	900,000
Combined (stratified–clustered)	30	900,000
Total	**90**	**2,700,000**

All sampled index sets preserve increasing order; therefore, each sampled subsequence retains the original relative order of elements in the full array.

6.4.3 Formation of Analysis Datasets

For downstream evaluation, full arrays and corresponding samples are grouped into two composite datasets:

- **Random-combined dataset:**
  pools all families without explicit run structure:

  $$\{\text{Permutation}\} \cup \{k\text{-pool, all } k\} \cup \{\text{Multiset, all } u\}.$$

  This yields:

  $$\text{9,000 full arrays} \quad \text{and} \quad \text{810,000 samples.}$$

- **Run-combined dataset:**
  pools all run-structured configurations across all three construction mechanisms and all values of R:

  $$\text{21,000 full arrays} \quad \text{and} \quad \text{1,890,000 samples.}$$

Table 7: Final analysis datasets: grouping of generated arrays and samples.

Dataset	Full Arrays	Samples
Random-combined	9,000	810,000
Run-combined	21,000	1,890,000
Total	**30,000**	**2,700,000**

Together, these two datasets provide complementary settings for evaluating whether sample-based presortedness measures can accurately recover global structure across both unstructured and run-dominated regimes.

6.5 Analytical Methodology

The analysis assesses how well sample-based presortedness measurements reproduce their full-array values across data families, sampling strategies, and sample sizes. The objective is to quantify both the *accuracy* of estimation (how close sample-based metrics are to their true values) and the *systematic effects* introduced by locality–coverage trade-offs during sampling.

Array–sample pairing and metric evaluation. For each experimental configuration, determined by the tuple

$$(n,\ m,\ \text{sampling method},\ \text{configuration}),$$

each full array A_i is paired with its corresponding index-ordered sample $A_i|_{S_i}$. For every normalized presortedness metric M, the full-array evaluations

$$x_i = M(A_i) \qquad \text{and} \qquad y_i = M(A_i|_{S_i})$$

form paired observations.

For each metric, the following statistical quantities are computed:

$$\bar{x},\ \bar{y},\quad s_x,\ s_y,\quad \text{bias} = \bar{y} - \bar{x},\quad \text{sd_ratio} = s_y/s_x,\quad r = \text{corr}_{\text{Pearson}}(x, y),$$

together with rank-based correlation ρ (Spearman) and Lin's concordance correlation coefficient (CCC). Small absolute bias, sd_ratio ≈ 1, and high correlation indicate that the sample-based metric preserves both the scale and the relative ordering of values across inputs.

To avoid unnecessary detail in the presentation of results, the discussion in Section *Experiment Results* focuses on three key indicators of estimation quality:

$$\text{bias},\quad \text{sd_ratio},\quad r\ (\text{Pearson}).$$

Scatter plots and tabulated summaries. For each metric and sampling configuration, scatter plots are generated with full-array values on the x-axis and sample-derived values on the y-axis, both normalized to $[0, 1]$, with the reference line $y = x$ shown for comparison. Tight alignment to the diagonal indicates high fidelity; systematic offsets correspond to bias or scale distortion.

Organization of aggregated results. Results are evaluated under two aggregated analysis views:

- **Random–combined:** all permutation, k-pool, and multiset generators pooled. This setting isolates effects driven primarily by value distribution (distinctness, frequency entropy), since positional order is random.
- **Run–combined:** all run-structured generators pooled across run counts. This view highlights positional order and its interaction with locality in sampling.

7 Experiment Results

This section reports empirical findings on the agreement between sample–based and full–array presortedness across the generation families and sampling designs defined earlier. Results are organized along two aggregate views: (i) *random–combined*, which pools inputs without explicit positional structure (uniform permutations, k–pool arrays, multiset permutations), and (ii) *run–combined*, which pools all run–structured inputs across run counts and construction mechanisms. This separation distinguishes value–diversity effects from positional–order effects and allows like–for–like comparisons of sampling behavior.

For each view, sample size m and the method–specific configuration are treated as experimental factors. The analysis considers two broad classes of metrics: *value–diversity* metrics (Distinct, Distinct Entropy, Value–Frequency Entropy) and *order–sensitive* metrics (Inversions, Runs, Removals, Oscillations, Displacement, Hamming, Run Entropy). In the random–combined view, order–sensitive metrics are uninformative by design and are therefore not emphasized; in the run–combined view, they become primary.

The following subsectionss discuss patterns method–by–method and metric–by–metric, highlight configurations that stabilize estimation, and summarize the trade–offs between locality and coverage.

7.1 Random-Combined: Results and Analysis

Across all three sampling methods, a clear *monotonic increase* in representativeness was observed as the sample size m increased. The Pearson correlation between metrics computed on the full arrays and their samples grows steadily with m, while the bias and dispersion ratio gradually decrease. This pattern is consistent for the three metrics we analyze in detail: **Distinct**, **Distinct Entropy**, and **Value–Frequency Entropy**. Importantly, because the random–combined corpus pools families *without positional structure* (uniform permutations, i.i.d. k-pool arrays, and multiset permutations), order–sensitive presortedness measures (inversions, runs, removals, oscillations, displacement, Hamming) exhibit only *negligible* array–sample correlations here; hence they are omitted from this section.

7.1.1 Cluster Sampling

Distinct. Correlations rise from r=0.609 at m=100 (2 log2) to r=0.897 at m=600 (sqrt). Bias decreases from ≈ 0.41 to ≈ 0.366, while *sd_ratio* increases from ≈ 0.053 to ≈ 0.192, reflecting tighter sample dispersion at small m and gradual convergence as m grows. Across m, strong configurations are moderate–length clusters: $\lfloor \log_2 m \rfloor$, $2 \cdot \lfloor \log_2 m \rfloor$, $\lfloor \sqrt{m} \rfloor$, and $2 \cdot \lfloor \sqrt{m} \rfloor$.

Distinct Entropy. The pattern is stronger: r climbs from 0.649 (2 log2, m=100) to 0.943 (sqrt, m=600), with small, stable bias ($\sim 0.068 \rightarrow 0.062$) and *sd_ratio* growing from ~ 0.056 to ~ 0.157. Best configs shift from log2/2 log2 at small/medium m to sqrt/2 sqrt at larger m.

Value–Frequency Entropy. Growth is milder: r moves from 0.600 (2 sqrt, m=100) to 0.859 (2 sqrt, m=600), with tiny bias ($\approx 0.014 \rightarrow 0.012$) and *sd_ratio* increasing from ≈ 0.102 to ≈ 0.277. Effective cluster lengths again sit in the log2/sqrt family; 2 log2 dominates at m=200-300, while sqrt/2 sqrt take over for $m \geq 400$.

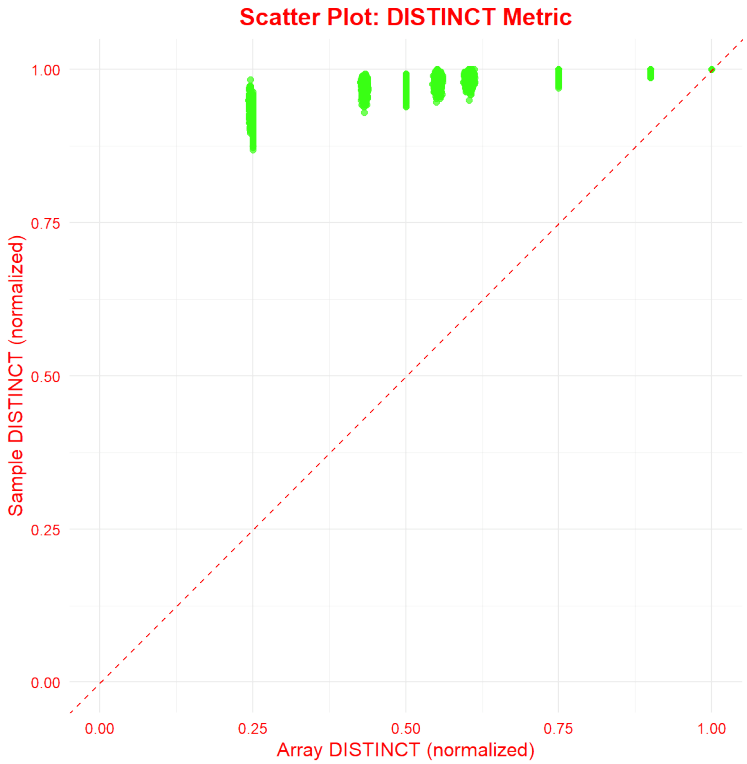


Figure 13: Distinct, cluster sampling, m=300, cluster length $cl = 2 \cdot \lfloor \log_2 m \rfloor$. Full-array values on the x-axis, sample values on the y-axis; dashed line y=x.

Interpretation of Fig. 13. The cloud lies tightly near the top ($y \approx 0.95$–1) while x spans a broad range ($\approx$ 0.15–0.95), i.e., the sample consistently overestimates distinctness (positive bias; for m = 300 we observed bias $\approx$ 0.391 and sd_ratio $\approx$ 0.108). This is expected in the random–combined view: distinctness in most arrays in this corpus is relatively high, while the sampling coverage in the experiments is small ($m/n < 0.06$). Under such low coverage, samples tend to include many values only once, inflating the estimated distinct element count relative to the full array.

7.1.2 Stratified Sampling

Distinct. Performance closely tracks cluster sampling: r grows from 0.613 ($n/20$, $m = 100$) to 0.896 (2 m, m=600), with similar bias and *sd_ratio* trajectories. Best strata are typically $n/20$ and $n/10$.

Distinct Entropy. Stratified slightly leads at small m (0.651 at m=100, $n/20$; 0.838 at m=200, *sl* = m) and remains statistically tied with cluster/combined for $m \geq 300$ (0.942 at m=600, 2 m). Bias is small and stable; *sd_ratio* follows the same smooth increase as cluster.

Value–Frequency Entropy. Stratified is slightly ahead at m=200–300 (0.768 for $n/20$; 0.823 for $n/10$) and remains very close to the other methods for m>400 ($\approx$ 0.841–0.859), showing the usual small bias and a gently increasing *sd_ratio*. Moderate strata ($n/20$, $n/10$) remain consistently strong.

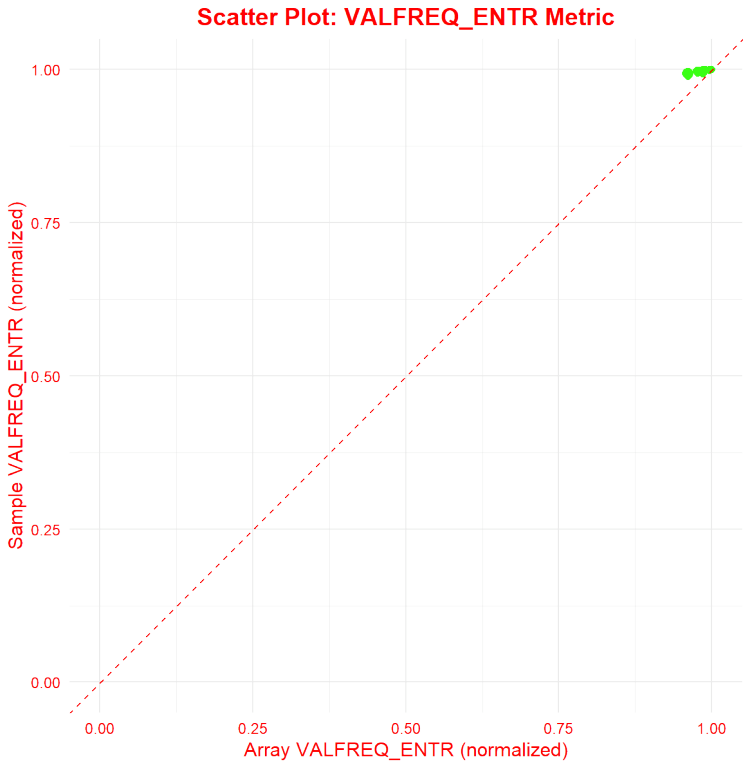


Figure 14: Value-Frequency Entropy, stratified sampling, s=300, stratum length $sl = n/20$.

Interpretation of Fig. 14. Points are concentrated in the upper-right corner ($x, y \approx 1$), reflecting nearly uniform frequency profiles across the pooled families (permutations have perfectly uniform frequencies; k-pool and multiset–permutation produce high-entropy profiles when k or u is large). The slight upward shift above the diagonal indicates a small positive bias (empirically ≈ 0.013 for m=300), with sd_ratio around 0.17.

7.1.3 Combined Sampling

Distinct. Combined stays very close to cluster/stratified across s (e.g., 0.607 at m=100; 0.897–0.897 neighbors at m=600), with best configs typically $n/20$ or $n/10$, and m at some budgets. Bias and *sd_ratio* match the trends above.

Distinct Entropy. Results are essentially tied with stratified for $m \geq 300$ (e.g., 0.896–0.942 across m=300–600), confirming that one contiguous block per stratum suffices to capture the distributional entropy in a position–independent corpus.

Value–Frequency Entropy. Combined alternates in the lead at large m (0.841 at m=400 with $n/10$, 0.860 at m=600 with $2\,m$), again with tiny bias and smoothly rising *sd_ratio*.

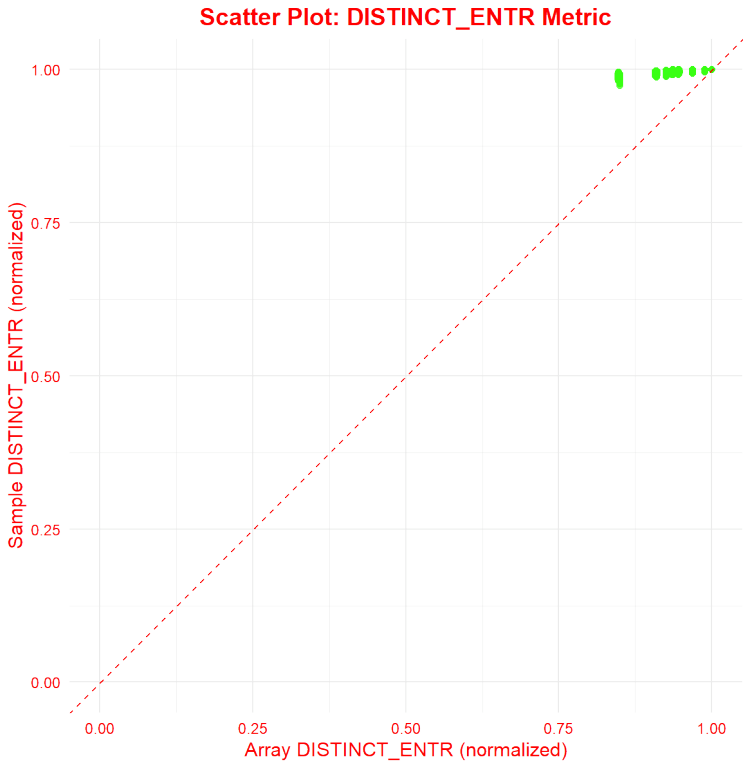


Figure 15: Distinct Entropy, combined sampling, m=300, stratum length $sl = n/20$.

Interpretation of Fig. 15. The cloud is extremely tight near $(1, 1)$ with a mild upward shift; this matches the statistics at m=300 (Pearson ≈ 0.896, bias ≈ 0.065, sd_ratio ≈ 0.095 for nearby configurations). This concentration follows directly from the generation design: the random–combined corpus mixes families whose value identities are independent of position; a single contiguous block per (moderately wide) stratum preserves local frequency structure, while covering many strata enforces global representativeness. Under these conditions, sample and array entropies align almost perfectly, with only a small positive bias introduced by reduced collision rates in finite samples.

Table 8: Best Pearson correlation r (and configuration) for each sample size m; random–combined corpus, **Distinct** metric.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.6094 (2log2 m)	0.6125 ($n/20$)	0.6071 (2 m)
200	0.7916 (log2 m)	0.7914 (m)	0.7887 (2 m)
300	0.8486 (2log2 m)	0.8459 ($n/20$)	0.8470 ($n/20$)
400	0.8742 (2log2 m)	0.8727 (m)	0.8728 ($n/10$)
500	0.8873 (2sqrt m)	0.8879 ($n/20$)	0.8860 (m)
600	0.8972 (sqrt m)	0.8963 (2 m)	0.8965 ($n/10$)

Table 8 commentary. For **Distinct**, *cluster* sampling attains the highest r for most m values. Exceptions are m=100 and m=500, where *stratified* is marginally better. *Combined* remains very close to the leaders at $m \geq 300$.

Table 9: Best Pearson correlation r (and configuration) for each sample size m; random–combined corpus, **Distinct Entropy** metric.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.6487 (2log2 m)	0.6514 ($n/20$)	0.6458 (2 m)
200	0.8372 (log2 m)	0.8379 (m)	0.8367 (m)
300	0.8957 (2log2 m)	0.8939 ($n/20$)	0.8956 ($n/20$)
400	0.9215 (2log2 m)	0.9215 (m)	0.9202 ($n/10$)
500	0.9342 (2sqrt m)	0.9347 ($n/20$)	0.9330 (m)
600	0.9425 (sqrt m)	0.9422 (2 m)	0.9418 ($n/10$)

Table 9 commentary. For **Distinct Entropy**, *stratified* leads at small sample sizes (m=100–200), the middle range is mixed (including a tie at m=400), and *cluster* achieves the top result at m=600. Overall differences between methods are very small for $m \geq 300$.

Table 10: Best Pearson correlation r (and configuration) for each sample size m; random–combined corpus, **Value-Frequency Entropy** metric.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.5996 (2sqrt m)	0.5952 (2 m)	0.5946 (2 m)
200	0.7649 (2log2 m)	0.7685 ($n/20$)	0.7630 ($n/20$)
300	0.8227 (2log2 m)	0.8229 ($n/10$)	0.8210 (2 m)
400	0.8402 (sqrt m)	0.8406 ($n/4$)	0.8408 ($n/10$)
500	0.8536 (log2 m)	0.8521 ($n/10$)	0.8523 ($n/4$)
600	0.8585 (2sqrt m)	0.8587 ($n/10$)	0.8599 (2 m)

Table 10 commentary. For **Value-Frequency Entropy** there is no single dominant method: *cluster* wins at m=100 and m=500, *stratified* is best at m=200–300, and *combined* leads at m=400 and m=600. All three remain very close across the board.

7.2 Summary.

Across the random–combined corpus, **strong correlations emerge at** $m \geq 300$ for the three value–diversity metrics (**Distinct**, **Distinct Entropy**, **Value–Frequency Entropy**). Below this budget the trends are present but weaker; by $m \approx 300$ the estimates stabilize, bias remains small, and *sd_ratio* moves smoothly toward 1.

The most effective configuration choices are:

- **Cluster sampling:** the consistently strongest configuration is a cluster size of $\lfloor 2log_2 m \rfloor$.
- **Stratified and Combined sampling:** the most reliable stratum widths are $sl = n/20$ and $sl = 2m$, offering a good balance between coverage across the array and local representativeness.

7.3 Run-Combined: Results and Analysis

Unlike the random–combined corpus, the *run–combined* corpus aggregates families with explicit positional structure—arrays are formed from runs using several generative schemes (noise–injection, sorted–base segmentation and permutation-segmentation), and then evaluated by the same sampling designs. Consequently, order–sensitive presortedness metrics (**Inversions**, **Runs**, **Removals**, **Oscillations**, **Displacement**, **Hamming**, **Run Entropy**) show *very strong* array–sample agreement whenever the sampling strategy preserves run boundaries or covers them evenly. For metrics that are additive or nearly additive over disjoint segments (*inversions*, *displacement*, *oscillations*), all three methods achieve near–perfect correlations ($r \approx 0.99$) with tiny bias and *sd_ratio* ≈ 1. Two noteworthy exceptions are **Runs** and **Hamming**: fixed–width stratification can fracture runs and overcount boundaries, yielding low r and large dispersion for *Runs*, while *Hamming* is inherently sensitive to pointwise misplacements, producing only moderate $r \in [0.55, 0.74]$ and large *sd_ratio*. **Run Entropy** remains medium–high ($r \approx 0.72$–0.80) but exhibits persistent negative bias and elevated variance because contiguous sampling inflates the share of short runs.

7.3.1 Cluster Sampling

Inversions. Near–perfect recovery across all budgets: r climbs from 0.994 (m=100, *log2 m*) to 0.9990 (m=600, *log2 m*). Bias is slightly negative and shrinks in magnitude ($-0.018 \rightarrow -0.004$), with *sd_ratio* close to 1 ($\sim$ 0.96–0.99). Best and most frequent winner: *log2 m*. The weakest variant is plain m at small m.

Runs. Excellent agreement when clusters remain long enough to include whole runs: r improves from 0.983 (m=100, m) to 0.997 (m=600, *2sqrt m*). Bias is small ($\approx$ 0–0.01) and *sd_ratio* ≈ 1. Best (and recurring) configuration for $m \geq 200$: *2sqrt m*.

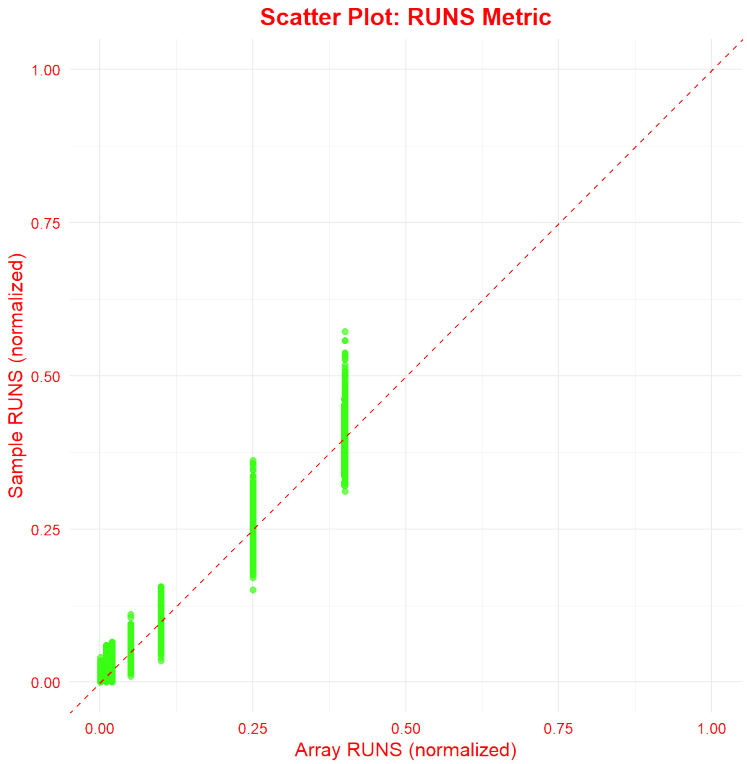


Figure 16: Runs, cluster sampling, m=200, cluster length $cl = 2 \cdot \lfloor \sqrt{m} \rfloor$. Full-array values on the x-axis, sample values on the y-axis; dashed line y=x.

Interpretation of Fig. 16. As the number of runs (large x) increases, points drift progressively *above* the diagonal, indicating a growing positive bias: clusters tend to cut runs and overcount boundaries in the sample. Despite this, correlation remains very high for clusters ($r \approx 0.9905$, bias ≈ 0.012, sd_ratio ≈ 0.99). For the same budget, the *combined* design with $n/4$ achieves a slightly better match ($r \approx 0.9940$) and smaller bias (≈ 0.006), although the gap is minor.

Removals. Strong but slightly behind stratified/combined at large s: r from 0.972 (m=100) to 0.996 (m=600), best with *log2 m* across budgets. Bias is negative and decreases in magnitude ($-0.098 \rightarrow -0.036$); *sd_ratio* increases toward 1 ($0.87 \rightarrow 0.95$).

Oscillations. Near–perfect and monotone in s: $r = 0.990$ (m=100, $cl = m$) to 0.997 (m=600, *2sqrt m*); bias ≈ 0 and *sd_ratio* ≈ 1. Winning configs are m for $m \leq 500$ and *2sqrt m* at m=600.

Displacement. Easiest to approximate: $r = 0.991$ (m=100) to 0.9984 (m=600), with negligible bias and *sd_ratio* ≈ 1. The repeated best choice is *log2 m* over all m.

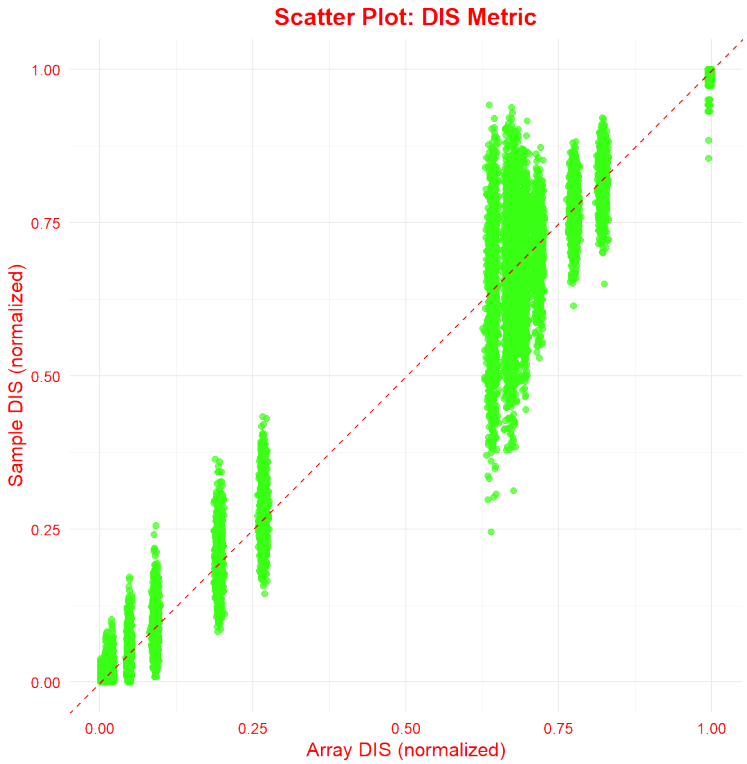


Figure 17: Displacement, cluster sampling, m=100, cluster length $cl = \lfloor \log_2 m \rfloor$.

Interpretation of Fig. 17. Points form a tight band along y=x, showing nearlinear agreement between sample and array displacement even at the smallest budget ($r \approx 0.9913$, bias $\approx$ 0.0003, sd_ratio $\approx$ 1.00). This reflects the additivity of displacement over contiguous blocks. Stratified and combined designs are marginally higher at this m (both $\approx$ 0.9975–0.9976), but the differences are negligible in practice.

Hamming. Moderate agreement only: r grows from 0.558 (m=100, *log2*) to 0.724 (m=600, *log2*). Bias is negative (sample underestimates accuracy) and large in absolute value at small m; dispersion is high (*sd_ratio* $\sim$ 6–9). Best configs alternate among *2log2*, *sqrt*, and *log2* as m increases.

Run Entropy. Mid–high correlations that decrease slightly with s (from 0.789 at m=100 to 0.716 at m=600). Bias is consistently negative ($-0.10 \rightarrow -0.053$), and *sd_ratio* is elevated ($\sim$ 4–6). Best (and frequent) configs: *log2* m for $m \leq 500$, shifting to *sqrt* at m=600.

7.3.2 Stratified Sampling

Inversions. Best overall among methods: $r = 0.9986$ at m=100 (m) up to 0.9998 at m=600 ($n/20$), with essentially zero bias and *sd_ratio* ≈ 1. Winning configurations repeat: m at small m, $n/20$ at large m.

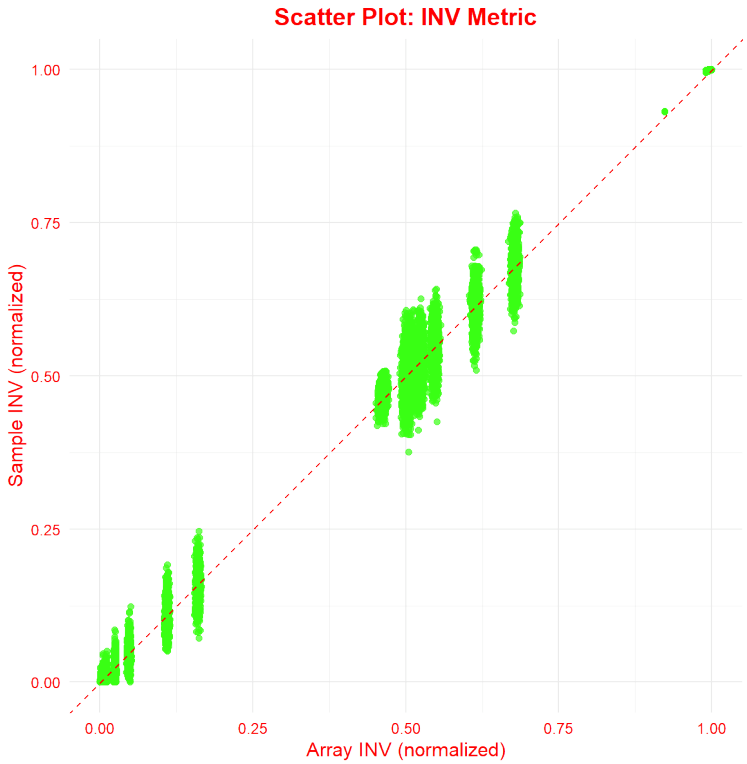


Figure 18: Inversions, stratified sampling, $m{=}\sqrt{n}{=}100$, stratum length $sl = m$ (one block per stratum of length m). Full-array values on the x-axis, sample values on the y-axis; dashed line $y{=}x$.

Interpretation of Fig. 18. Points concentrate tightly along the diagonal with only a slight upward shift, indicating nearperfect recovery of inversion counts under stratified sampling at this budget (Pearson $r{\approx}0.9986$, tiny positive bias $\sim$0.0015, and *sd_ratio* $\sim$1.005 from the summary). The nearlinear cloud reflects the additivity of inversions across strata: evenly covering the array with strata of size sl preserves the segment structure and yields sample estimates that track the fullarray almost onetoone.

Runs. Sensitive to stratum boundaries: r is low and increases only modestly with m (from 0.190 at $m{=}100$ to 0.636 at $m{=}600$). Dispersion is inflated (*sd_ratio* $\sim$2.05–2.42) and bias strongly positive due to overcounted run breaks. Best results occur with wider strata (e.g., $n/4$ or $2\,m$), but correlations remain far below cluster/combined.

Removals. Top performer: r moves from 0.992 ($m{=}100$, $n/4$) to 0.999 ($m{=}600$, m). Bias magnitude steadily decreases and *sd_ratio* approaches 1. Robust winners: $n/4$ and $2\,m$ (mid–large s), m at the largest m.

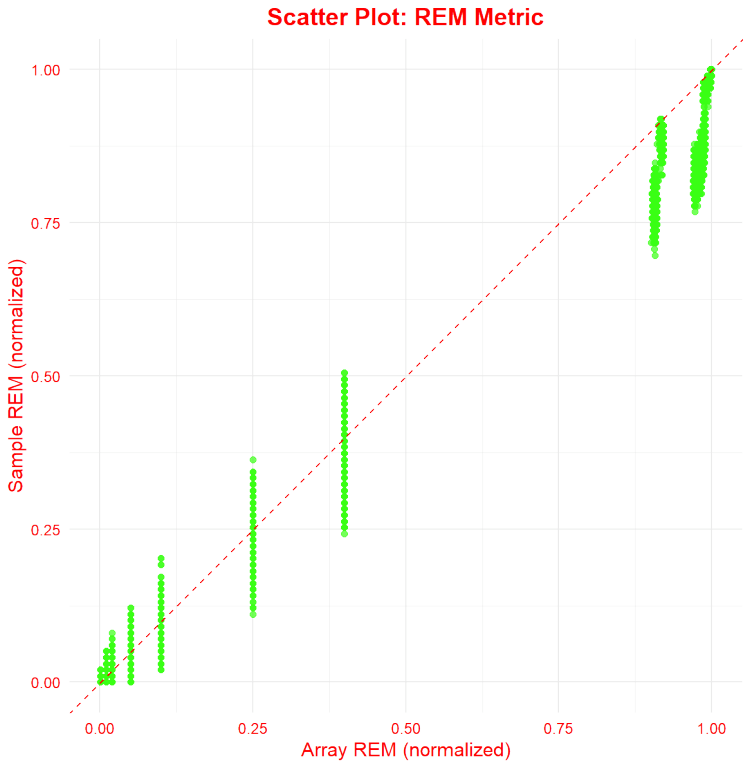


Figure 19: Removals, stratified sampling, $m{=}\sqrt{n}{=}100$, stratum length $sl = n/4$. Full-array values on the x-axis, sample values on the y-axis; dashed line $y{=}x$.

Interpretation of Fig. 19. The cloud hugs the diagonal but sits slightly below it for most of the range, showing an *underestimation* of removals by the sample (summary stats at this setting: $r{\approx}0.9919$, bias ≈ -0.047, *sd_ratio* ≈ 0.94). This pattern matches the expectation that fixedwidth strata can smooth over small boundary effects, yielding very high correlation with a small negative bias.

Oscillations. Unreliable due to fragmentation: best r ranges only 0.19–0.35 as m grows, with persistent positive bias and *sd_ratio* departing from 1. The least–bad configurations are $n/4$ (small/medium s) and $2\,m$ (large m).

Displacement. Marginally best overall: r from 0.9975 ($m{=}100$) to 0.9995 ($m{=}500$–600), negligible bias, *sd_ratio* ≈ 1. Stable winners: $sl = m$ (for most m) and $n/20$ at $m \in \{300, 600\}$.

Hamming. Moderate but competitive within this metric: r increases from 0.558 ($m{=}100$, $n/10$) to 0.728 ($m{=}600$, $n/20$). Bias is negative and dispersion high (*sd_ratio* $\sim$6–9). Best recurring strata: $n/4$ for $m \leq 400$, $n/20$ thereafter.

Run Entropy. Slightly best (on average) among methods: r in the 0.79–0.73 band as m grows, with negative bias and elevated *sd_ratio*. Preferred strata concentrate around $n/20$ (small/medium m) and $n/10$ (large m).

7.3.3 Combined Sampling

Inversions. Near–perfect and stable: $r = 0.9987$ ($m{=}100$, $sl = m$) to 0.9982 ($m{=}600$, $n/20$), with tiny negative bias and *sd_ratio* ≈ 1. Best configs repeat: $sl = m$ up to $m{=}500$, $n/20$ at $m{=}600$.

Runs. *Best overall* for this metric: r improves from 0.986 ($m{=}100$) to 0.998 ($m{=}600$), consistently

outperforming cluster and vastly exceeding stratified. The winning and most frequent configuration is $n/4$, i.e., one wide contiguous block per stratum, which avoids boundary overcounts.

Removals. Strong and close to stratified at small/medium m: $r \approx 0.991$ at m=100–300 (m) and up to 0.9913 at m=600 ($n/20$). Bias magnitudes are modest and *sd_ratio* close to 1. Typical winners: m (most m), $n/20$ at the largest m.

Oscillations. *Best overall* across budgets: $r = 0.986$ (m=100) to 0.998 (m=600). The dominant configuration is $n/4$ for all m, again reflecting that a single contiguous block per (wide) stratum preserves alternation structure without adding spurious boundaries.

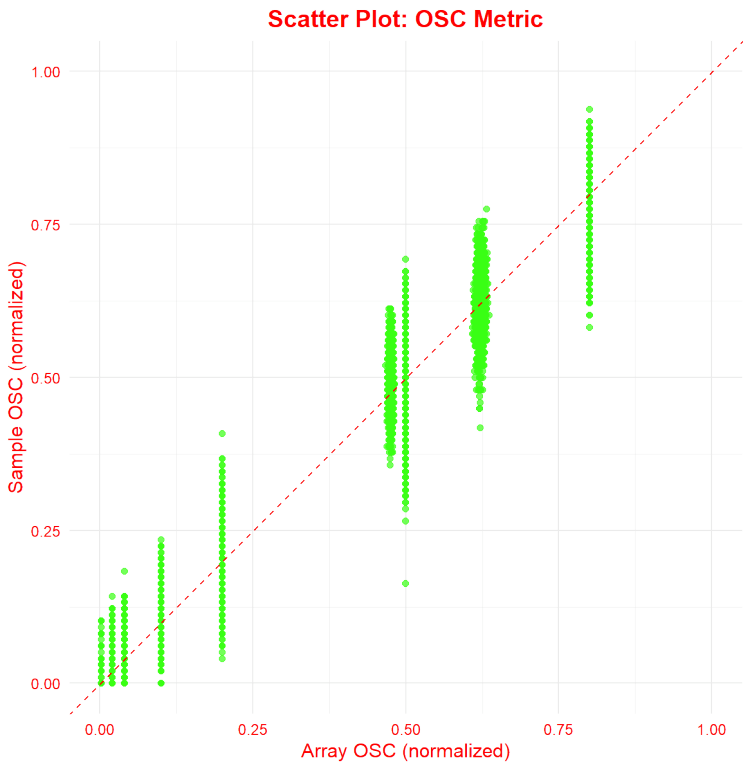


Figure 20: Oscillations, combined sampling, m=100, stratum length $sl = n/4$. Full-array values on the x-axis, sample values on the y-axis; dashed line y=x.

Interpretation of Fig. 20. The scatter points follow the diagonal closely but show a consistent upward offset, indicating that oscillation counts are *overestimated* by the sample. This arises because strata of length $n/4$ may capture short alternating fragments, which inflates local oscillation density relative to the full array. The degree of overestimation decreases noticeably as the sample size increases: it is still evident at m=100, but becomes much smaller by $m \geq 300$. The tight vertical clustering of points indicates strong correlation ($r \approx 0.985$ at m=100) and shows that combined sampling effectively balances local continuity with global run coverage.

Displacement. Extremely high agreement, slightly below stratified: $r = 0.9976$ at m=100 (*smlLength*) and 0.9973 at m=600 ($n/20$). Bias and *sd_ratio* are negligible. Best configs: $sl = m$ (small/medium m), $n/20$ (large m).

Hamming. Comparable to other methods and *best at the largest budget*: r rises to 0.735 at m=600 (m), slightly topping cluster/stratified. Bias remains negative; dispersion is high. Best recurring choices: $n/20$ at m=200–300, m at $m \geq 500$.

Run Entropy. Competitive with stratified at small m (0.801 at m=100, $2\,m$) and stabilizing near 0.725 at m=600 (m). Bias is negative and *sd_ratio* elevated; best configs repeat: m and $n/20$.

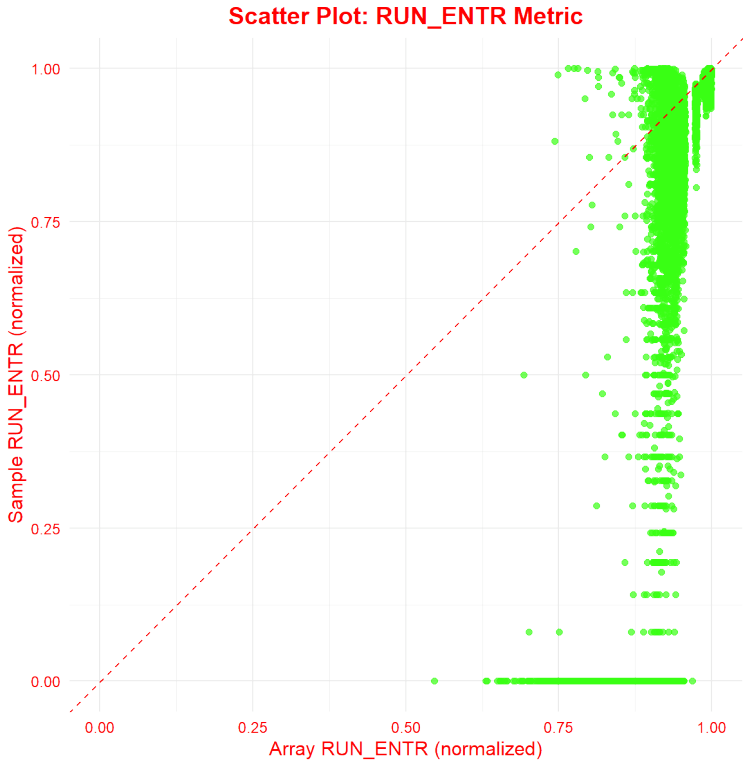


Figure 21: Run Entropy, combined sampling, m=100, stratum length $sl = 2m$. Full-array values on the x-axis, sample values on the y-axis; dashed line y=x.

Interpretation of Fig. 21. Most arrays in the run–combined corpus have high normalized run entropy, since they are constructed by concatenating multiple monotone runs of comparable length whose boundaries are distributed throughout the array. This produces the concentration of full-array values in the range $x \approx 0.8$–1.0.

Within this regime, the sample-based estimates form a noticeably wider vertical spread. The key reason is that a small sample generally contains elements from only a subset of the runs present in the full array, and the relative proportions of run lengths in the sample need not match the global distribution. This produces the characteristic triangular cloud slightly below the diagonal: *the overall ranking of arrays by entropy is preserved*, but the absolute values tend to be underestimated when sample coverage captures fewer run transitions. The correlation remains strong ($r \approx 0.80$).

Table 11: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Inversions** (inv).

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.9941 (log2 m)	0.9986 (m)	0.9987 (m)
200	0.9972 (log2 m)	0.9993 (m)	0.9985 (m)
300	0.9981 (log2 m)	0.9995 ($n/20$)	0.9983 (m)
400	0.9987 (log2 m)	0.9996 (m)	0.9983 (m)
500	0.9989 (log2 m)	0.9997 ($n/20$)	0.9981 (m)
600	0.9990 (log2 m)	0.9998 ($n/20$)	0.9982 ($n/20$)

Table 11 commentary. All methods perform nearly perfectly for **Inversions**. *Stratified* shows a marginal lead overall (especially with $n/20$ or m), while *cluster* with *log2 m* and *combined* are statistically indistinguishable at large m.

Table 12: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Runs** (runs).

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.9826 (m)	0.3617 ($n/4$)	0.9862 ($n/4$)
200	0.9905 (2sqrt m)	0.4519 ($n/4$)	0.9940 ($n/4$)
300	0.9937 (2sqrt m)	0.5125 ($n/4$)	0.9957 ($n/4$)
400	0.9951 (2sqrt m)	0.5615 (2 m)	0.9968 ($n/4$)
500	0.9960 (2sqrt m)	0.5961 ($n/4$)	0.9973 ($n/4$)
600	0.9966 (2sqrt m)	0.6363 (2 m)	0.9978 ($n/4$)

Table 12 commentary. Combined sampling is clearly the best for **Runs**, with the optimal and most stable configuration $n/4$. *Cluster* is consistently close, while *stratified* performs poorly due to boundary fragmentation.

Table 13: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Removals** (rem).

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.9715 (log2 m)	0.9919 ($n/4$)	0.9915 (m)
200	0.9852 (log2 m)	0.9961 ($n/4$)	0.9908 (m)
300	0.9900 (log2 m)	0.9976 (2 m)	0.9900 (m)
400	0.9931 (log2 m)	0.9983 ($n/4$)	0.9903 (m)
500	0.9947 (log2 m)	0.9987 (2 m)	0.9902 (m)
600	0.9955 (log2 m)	0.9989 (m)	0.9913 ($n/20$)

Table 13 commentary. For **Removals**, *stratified sampling* achieves the highest correlations across all m, especially with $n/4$ or $2\,m$. Starting from $m \geq 400$, all three methods become nearly identical ($r \approx 0.99$–1.0), reflecting full convergence of sample and array removal counts.

Table 14: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Oscillations** (osc).

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.9898 (m)	0.1898 (m)	0.9859 ($n/4$)
200	0.9940 (m)	0.1564 ($n/4$)	0.9943 ($n/4$)
300	0.9955 (m)	0.1956 ($n/4$)	0.9964 ($n/4$)
400	0.9966 (m)	0.2428 ($n/4$)	0.9974 ($n/4$)
500	0.9969 (m)	0.2906 ($n/4$)	0.9979 ($n/4$)
600	0.9973 (2sqrt m)	0.3506 (2 m)	0.9983 ($n/4$)

Table 14 commentary. **Oscillations** show the highest accuracy under *combined sampling*, with the best configuration $n/4$. *Cluster* matches closely, while *stratified* remains unstable due to excessive run boundary segmentation.

Table 15: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Displacement**.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.9913 (log2 m)	0.9975 (m)	0.9976 (m)
200	0.9955 (log2 m)	0.9987 (m)	0.9975 (m)
300	0.9970 (log2 m)	0.9991 ($n/20$)	0.9973 (m)
400	0.9978 (log2 m)	0.9993 (m)	0.9973 (m)
500	0.9982 (log2 m)	0.9995 (m)	0.9972 (m)
600	0.9984 (log2 m)	0.9995 ($n/20$)	0.9973 ($n/20$)

Table 15 commentary. All three designs perform almost identically for **Displacement** ($r > 0.99$). *Stratified sampling* remains marginally best, particularly with m or $n/20$, confirming its consistency for additive metrics.

Table 16: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Hamming**.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.5577 (log2 m)	0.5577 ($n/10$)	0.5556 (2 m)
200	0.6321 (2log2 m)	0.6270 ($n/4$)	0.6296 ($n/20$)
300	0.6709 (sqrt m)	0.6709 ($n/4$)	0.6729 ($n/20$)
400	0.7020 (sqrt m)	0.7035 ($n/4$)	0.6965 (2 m)
500	0.7201 (2log2 m)	0.7172 (2 m)	0.7219 (m)
600	0.7235 (log2 m)	0.7279 ($n/20$)	0.7354 (m)

Table 16 commentary. For **Hamming**, all correlations remain moderate, but *combined sampling* attains the best result at m=600 (m). Performance steadily improves with m, though the metric's pointwise nature limits absolute accuracy.

Table 17: Best Pearson correlation r (and configuration) for each sample size m; run–combined corpus, **Run Entropy**.

m	Cluster: r (config)	Stratified: r (config)	Combined: r (config)
100	0.7886 (log2 m)	0.7993 ($n/4$)	0.8013 (2 m)
200	0.7676 (log2 m)	0.7924 ($n/20$)	0.7829 (m)
300	0.7582 (log2 m)	0.7747 ($n/20$)	0.7614 (m)
400	0.7490 (log2 m)	0.7640 ($n/10$)	0.7548 ($n/20$)
500	0.7360 (2log2 m)	0.7441 ($n/10$)	0.7256 ($n/20$)
600	0.7159 (sqrt m)	0.7269 ($n/10$)	0.7253 (m)

Table 17 commentary. For **Run Entropy**, correlations gradually *decrease* as the sample size increases, because larger contiguous samples tend to capture longer, more homogeneous sequences of runs. This reduces the diversity of short runs and thus diminishes the entropy contrast between the sample and the full array. The highest overall result is observed at the smallest sample size (m=100), where *combined sampling* with configuration $2\,m$ achieves the best correlation among all methods. At larger m, correlations converge and differences between sampling strategies become minimal.

7.4 Summary.

Across all order–sensitive metrics in the run–combined corpus, **all three sampling methods already achieve strong agreement at the smallest budget** m=100. Even at 1% coverage of the array, correlations for **Runs**, **Inversions**, **Displacement**, **Removals**, and **Oscillations** are high ($r \gtrsim 0.98$) with low bias and *sd_ratio* close to 1. Increasing the sample size to $m\approx$300 further stabilizes the estimates, but improvements beyond that point are comparatively small for most metrics.

The most effective configurations follow a clear pattern:

- **Cluster sampling:** best performance is consistently achieved with $\lfloor \log_2 m \rfloor$ and $2 \cdot \lfloor \log_2 m \rfloor$–sized clusters, which provide balanced locality while ensuring broad positional coverage.
- **Stratified and Combined sampling:** the strongest and most stable choices are $sl = m$ and $sl = n/4$, as these strata widths preserve run continuity without introducing excessive fragmentation.

Thus, **moderate locality**—either through logarithmic–scale clusters or medium–width strata—provides the best trade–off between capturing local structure and maintaining global representativeness.

8 Discussion

8.1 Overview

The observed behavior of sampling accuracy across metrics can be directly explained by the structural properties of the underlying data generation methods. In the *run–combined* corpus, the positional layout of values is highly organized: arrays are composed of runs with controlled boundary patterns and predictable variation. As a result, many presortedness metrics can be recovered accurately from relatively small samples, provided that the sampling method preserves run continuity.

In contrast, the *random–combined* corpus contains families where positional structure is intentionally absent and only the *global value distribution* carries information. Here, sampling accuracy depends primarily on how well the sample reflects frequency diversity rather than order structure.

8.2 Structure–Sensitive Metrics (Run–Combined)

In the run–combined corpus, arrays are constructed from monotone segments whose lengths follow approximately multinomial distributions (e.g., in the sorted–base segmentation and permutation–segmentation models). As a result, run lengths tend to be relatively balanced, and the overall structural pattern repeats regularly across the sequence. When the number of runs is sufficiently large, this regularity greatly increases the likelihood that even small samples contain a representative cross–section of the underlying structural pattern.

For metrics that depend primarily on the placement and relative proportions of run boundaries—such as **Runs**, **Run Entropy**, **Oscillations**, and **Removals**—this repeated structure makes accurate estimation possible at small sampling scales. Among the sampling strategies, the **combined** method consistently yields the highest accuracy on these metrics. Its effectiveness stems from the interplay of stratification (which enforces consistent coverage across the global run layout) and clustering (which preserves local continuity, ensuring that the internal coherence of runs is reflected in the sample). In settings where the array contains many comparably sized runs, this hybrid preservation of global spread and local contiguity is particularly advantageous.

In contrast, metrics such as **Inversions** and **Displacement** depend less on precisely locating run boundaries and more on capturing broad ordering trends across the array. For these metrics, **stratified** sampling performs especially well. Because it distributes sample points evenly across the full index range, it captures large–scale value monotonicity even when internal run boundaries are not sampled directly. Thus, while combined sampling excels on boundary–dependent metrics, stratified sampling provides strong performance on order–trend–based metrics by systematically sampling the array as a whole.

8.3 Duplicate–Sensitive Metrics (Random–Combined)

In the random–combined corpus, positional ordering carries little or no information; the dominant source of structure lies in the *value–frequency distribution*. Since duplicates are globally intermixed and no contiguous region is inherently more or less informative than another, preserving local continuity offers no specific advantage. Consequently, **cluster** and **combined** sampling do not provide benefits over **stratified** sampling in this setting.

Because all three methods used here are non–adaptive (i.e., they do not detect or target regions of higher duplicate concentration), sampling quality depends primarily on systematic coverage of the full index range. Under this condition, all three strategies converge toward accurate estimates as sample size grows. This behavior is reflected in the steady improvement in correlations for metrics driven by value diversity, such as **Distinct Count**, **Distinct Entropy**, and **Value–Frequency Entropy**. Once the sample reaches approximately $m \geq 300$ (3% of the array), all methods achieve strong agreement with the full–array metric values, and differences between sampling strategies become negligible.

9 Conclusion

This work examined the feasibility of estimating presortedness from partial observations of data, with the goal of determining whether small, order-preserving samples can provide reliable guidance for algorithm selection in large-scale sorting contexts. By separating positional structure from value distribution in the data-generation process, and by evaluating multiple sampling strategies across a range of presortedness measures, the study clarifies which structural properties of input arrays are recoverable from small samples and under what conditions.

The results show that many presortedness metrics can be approximated from sampled data, provided that the sample retains the essential features relevant to the metric. In the data models examined in this work, metrics tied to run layout or overall ordering can be recovered with high accuracy from relatively small samples, while metrics driven primarily by value frequency require larger samples to stabilize. These observations reflect the limitations of the controlled data models and sampling methods considered, and do not imply universal guarantees across arbitrary real-world inputs.

Taken together, the findings indicate that presortedness measurement does not inherently require full inspection of the input data. Under appropriate conditions, lightweight sampling can supply sufficient information to guide adaptive sorting strategies in large-scale environments.

10 Future Work

The results of this study clarify when small samples can provide reliable estimates of presortedness and where this approach begins to lose effectiveness. These limitations point to several natural directions for further work, focusing on expanding data variability, evaluating sampling behavior across scales, and incorporating adaptive sampling mechanisms.

10.1 Data Generation Extensions

The run–structured families used in this work generate arrays where run lengths follow a multinomial allocation: runs differ in length, but the overall distribution is relatively balanced and does not include extreme scale disparities. This design keeps the structural variation controlled and ensures that no single run dominates the array. In many real datasets, run lengths follow highly non–uniform distributions. Incorporating such distributions would make it possible to evaluate how sampling performance depends on the heterogeneity of local sortedness rather than solely on the number of runs.

Additionally, the run–combined corpus used here is largely duplicate–free (except in the noise–injection variant). In contrast, many practical arrays contain a mixture of sorted segments and repeated values. Extending the run generators to allow controlled value reuse within and across runs would enable joint examination of *positional structure* and *value multiplicity*.

10.2 Sampling Performance under Varying Dataset Sizes

The experiments in this study were conducted using a fixed array length. While this allows controlled comparison across sampling configurations and data families, it does not reveal how sampling accuracy behaves as the size of the input varies. A natural extension is to evaluate the stability of the observed relationships under varying array sizes.

Such scaling analysis would clarify whether the conclusions drawn here generalize across different problem sizes, and would indicate whether sample-based presortedness estimation remains reliable in

both small in-memory settings and large-scale processing contexts.

10.3 Adaptive Sampling

All sampling strategies considered in this study are *non–adaptive*: the sampling pattern is determined solely by global parameters rather than by the structure of the input array. While this design isolates sampling geometry effects cleanly, it does not leverage information available during sampling itself. A natural direction for future work is the development of *content–aware* sampling procedures.

In summary, extending data variability, introducing adaptive sampling behavior, and embedding presortedness estimation into downstream algorithmic workflows represent promising next steps in advancing sample-based analysis of partial order.

A Appendix A: Pseudocode for Presortedness Measures

This appendix provides reference pseudocode for all presortedness measures used in the study. Implementations follow the definitions from Section § Preliminaries and match the code in evaluation framework.

A.1 Number of Runs (R)

```
procedure NumberOfRuns(a):

    # Preconditions:
    #   a is an array (possibly empty)

    n <- length(a)
    if n == 0 then return 0

    R <- 1
    for i <- 1 to n-1 do
        if a[i] < a[i-1] then
            R <- R + 1
        end if
    end for
    return R
```

A.2 Run-length Entropy ($H_{\mathbf{runs}}$)

```
procedure RunLengthEntropy(a):

    # Preconditions:
    #   a is an array (possibly empty)

    n <- length(a)
    if n == 0 then return 0.0

    # collect run lengths for strictly increasing runs
    lengths <- empty list
    cur <- 1
    for i <- 1 to n-1 do
        if a[i] < a[i-1] then
            append(lengths, cur)
            cur <- 1
        else
            cur <- cur + 1
        end if
    end for
    append(lengths, cur)
```

```
    # H = - (r/n) * ln(r/n)
    H <- 0.0
    for each r in lengths do
        p <- r / n
        H <- H - p * ln(p)
    end for
    return H
```

A.3 Inversions (Inv)

```
procedure CountInversions(a):

    if length(a) < 2 then return 0
    tmp <- new array of length length(a)
    return MergeSortAndCount(a, tmp, 0, length(a)-1)


procedure MergeSortAndCount(a, tmp, L, R):

    if L >= R then return 0
    M <- floor((L + R) / 2)

    x <- MergeSortAndCount(a, tmp, L,   M)
    y <- MergeSortAndCount(a, tmp, M+1, R)
    z <- MergeAndCount(a, tmp, L, M, R)

    return x + y + z


procedure MergeAndCount(a, tmp, L, M, R):

    i <- L
    j <- M + 1
    k <- L
    inv <- 0

    while i <= M and j <= R do
        if a[i] <= a[j] then
            tmp[k] <- a[i];  i <- i + 1
        else
            tmp[k] <- a[j];  j <- j + 1
            inv <- inv + (M - i + 1)
        end if
        k <- k + 1
    end while

    while i <= M do tmp[k] <- a[i]; i <- i + 1; k <- k + 1 end while
    while j <= R do tmp[k] <- a[j]; j <- j + 1; k <- k + 1 end while
```

```
    for t <- L to R do a[t] <- tmp[t] end for
    return inv
```

A.4 Removals to Sort ($\mathrm{Rem} = n - \mathrm{LIS}_{\uparrow}$)

```
procedure RemovalsToSort(a):

    # LIS of non-decreasing subsequences (uses upper_bound)

    n <- length(a)
    if n == 0 then return 0

    tails <- empty array

    for x in a do
        # find first position with tails[pos] > x
        pos <- first index in [0 .. length(tails)] where tails[pos] > x
        if pos == length(tails) then
            append(tails, x)
        else
            tails[pos] <- x
        end if
    end for

    LIS <- length(tails)
    return n - LIS
```

A.5 Oscillation Count (Osc)

```
procedure OscillationCount(a):

    n <- length(a)
    if n < 3 then return 0

    cnt <- 0
    for i <- 1 to n-2 do
        isPeak   <- (a[i-1] < a[i] and a[i] > a[i+1])
        isValley <- (a[i-1] > a[i] and a[i] < a[i+1])
        if isPeak or isValley then
            cnt <- cnt + 1
        end if
    end for
    return cnt
```

A.6 Spearman Footrule Displacement (Dis)

```
procedure SpearmanFootrule(a):

    n <- length(a)
    if n == 0 then return 0

    # Stable sort of pairs (value, original_index)
    s <- array of pairs (a[i], i) for i=0..n-1
    stable_sort s by (value ascending, original_index)

    # For each value v, queue target positions in sorted order
    Q <- map: value -> empty queue
    for t <- 0 to n-1 do
        v <- s[t].value
        enqueue(Q[v], t)
    end for

    # Sum |i - (i)| by consuming queues in the original order
    sum <- 0
    for i <- 0 to n-1 do
        v <- a[i]
        t <- front(Q[v]); dequeue(Q[v])
        sum <- sum + abs(i - t)
    end for
    return sum
```

A.7 Hamming Distance to Sorted Order (Ham)

```
procedure HammingToSorted(a):

    n <- length(a)
    if n == 0 then return 0

    s <- copy of a
    stable_sort s ascending

    cnt <- 0
    for i <- 0 to n-1 do
        if a[i] != s[i] then
            cnt <- cnt + 1
        end if
    end for
    return cnt
```

A.8 Distinct Count (U)

```
procedure DistinctCount(a):
```

```
    S <- empty hash set
    for x in a do
        insert S, x
    end for
    return size(S)
```

A.9 Value-frequency Entropy ($H_{\mathbf{freq}}$)

```
procedure ValueFrequencyEntropy(a):

    n <- length(a)
    if n == 0 then return 0.0

    freq <- empty hash map  # value -> count
    for x in a do
        freq[x] <- freq.get(x, 0) + 1
    end for

    H <- 0.0
    for each (v, f) in freq do
        p <- f / n
        H <- H - p * ln(p)
    end for
    return H
```

A.10 Distinct-element (Hartley) Entropy (H_0)

```
procedure HartleyEntropy(a):

    U <- DistinctCount(a)
    if U <= 1 then return 0.0
    return ln(U)
```

A Appendix B: Helper Routines for Data Generation

This appendix provides auxiliary procedures used by the data–generation methods in Section 4. These routines define common substeps such as constructing positive compositions, selecting non-adjacent indices, computing run boundaries, and assembling run segments.

B.1 RandomPositiveProfile (positive composition of n into u parts)

```
procedure RandomPositiveProfile(n, u):

    # Preconditions:
    #   n > 0, u >= 1, n >= u
```

```
    if u == 1 then
        return array [n]

    # Choose u-1 divider positions uniformly from {0, ..., n-2}
    k <- u - 1
    J <- SampleWithoutReplacement(n - 1, k)   # uniform without replacement
    sort J increasing

    x <- new array of length u
    x[0] <- (J[0] + 1)

    for i <- 1 to k - 1 do
        x[i] <- (J[i] + 1) - (J[i - 1] + 1)
    end for

    x[u - 1] <- n - (J[k - 1] + 1)
    return x
```

B.2 ChooseNonAdjacentInRange

```
procedure ChooseNonAdjacentInRange(lo, hi, k):

    # Choose k indices in [lo..hi] with no two adjacent.
    # Feasible capacity = ceil((hi - lo + 1) / 2).

    if k == 0 then return empty list

    # First candidate pattern: lo, lo+2, ...
    S1 <- empty list
    for i <- lo to hi step 2 do
        append S1, i
    end for

    # Second pattern if S1 is too small
    S2 <- empty list
    for i <- lo+1 to hi step 2 do
        append S2, i
    end for

    # Select feasible pattern
    C <- (|S1| >= k) ? S1 : S2

    # Uniformly reduce size to k if needed
    while |C| > k do
        j <- UniformInteger(0, |C|-1)
        remove C[j]
    end while
    return C
```

B.3 BuildRunLengths (positive run sizes summing to n)

```
procedure BuildRunLengths(n, R):

    # Preconditions: 1 <= R <= n

    L <- new array of length R, initialized to 1
    extra <- n - R

    # Unbiased distribution in rounds:
    while extra > 0 do
        r <- min(extra, R)
        P <- SampleWithoutReplacement(R, R)   # random permutation of 0..R-1
        for t <- 0 to r - 1 do
            j <- P[t]
            L[j] <- L[j] + 1
        end for
        extra <- extra - r
    end while
    return L
```

B.4 BuildRunStarts (prefix sums of run lengths)

```
procedure BuildRunStarts(L):

    R <- length(L)
    S <- new array of length R
    cur <- 0
    for r <- 0 to R - 1 do
        S[r] <- cur
        cur <- cur + L[r]
    end for
    return S
```

B.5 SortEachSegmentAscending (in-place sort of each run)

```
procedure SortEachSegmentAscending(A, S, L):

    for r <- 0 to |L|-1 do
        s <- S[r]
        e <- s + L[r]
        StableSortAscending(A[s .. e-1])
    end for
```

B.6 ComputeSegmentsMeta (store start/length/min/max)

```
procedure ComputeSegmentsMeta(A, S, L):
```

```
    Seg <- empty list
    for r <- 0 to |L|-1 do
        start <- S[r]
        len   <- L[r]
        minv  <- A[start]
        maxv  <- A[start + len - 1]
        append (start, len, min=minv, max=maxv) to Seg
    end for
    return Seg
```

B.7 OrderByMaxDescending (stable ordering of segments)

```
procedure OrderByMaxDescending(Seg):

    StableSortByKey(Seg, key = 'max', order = descending)
```

B.8 ConcatBySegments (assemble final array)

```
procedure ConcatBySegments(src, Seg):

    out <- empty list
    for each s in Seg do
        for i <- s.start to s.start + s.len - 1 do
            append out, src[i]
        end for
    end for
    return out
```

Bibliography

Bentley, J. and McIlroy, D. (1993). Engineering a sort function. In *Software—Practice and Experience*, volume 23, pages 1249–1265.

Cook, R. and Kim, D. (1980). Best sorting algorithm for nearly sorted lists. *Communications of the ACM*, 23(11):620–624.

Dewar, R., Merritt, S., and Sharir, M. (1982). Some modified algorithms for dijkstra's longest upsequence problem. *Acta Informatica*, 18:1–15.

Dijkstra, E. W. (1980). Some beautiful arguments using mathematical induction. *Acta Informatica*, 13:1–13.

Estivill-Castro, V., Mannila, H., and Wood, D. (1993). Right invariant metrics and measures of presortedness. *Discrete Applied Mathematics*, 42(3):251–264.

Estivill-Castro, V. and Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476.

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching.* Addison-Wesley.

Levcopoulos, C. and Petersson, O. (1993). Heapsort—adapted for presorted files. In *Proceedings of WADS*, pages 499–509.

Mannila, H. (1985). Measuring the degree of presortedness in data sets. *Acta Informatica*, 22:375–393.

Mehlhorn, K. (1979). Sorting presorted files. In *Proceedings of the 4th GI Conference on Theoretical Computer Science*, pages 199–212. Springer.

Munro, A. and Wild, S. (2018). Powersort: Efficiently detecting and exploiting runs in real-world data. In *Proceedings of ESA*.

Peters, T. (2002). Timsort: A stable natural mergesort. `https://bugs.python.org/file4451/timsort.txt`.

Peters, T. (2004). Timsort implementation in cpython. `https://github.com/python/cpython/blob/main/Objects/listsort.txt`.

Petersson, O. and Moffat, A. (1995). A framework for adaptive sorting. *Algorithmica*, 13:1–22.

Sedgewick, R. (1998). Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857.

Sedgewick, R. and Bentley, J. (2002). Quicksort is optimal (slides). `http://www.cs.princeton.edu/~rs/talks/QuicksortIsOptimal.pdf`.

Wild, S. (2017). Quicksort is optimal for many equal keys. *SIAM Journal on Computing*, 46(2):669–706.